Towards Resource-Efficient Training-as-a-Service by Exploiting Runtime Gradient Statistics

Guanjie Wang, *Student Member, IEEE,* Chen Chen, *Member, IEEE,* Yifei Liu, *Student Member, IEEE,* Qiang Wang, *Member, IEEE,* Cong Wang, *Fellow, IEEE,* Quan Chen, *Member, IEEE,* Minyi Guo, *Fellow, IEEE*

***Abstract*—The recent success of artificial intelligence motivates many non-professional users to train their own models; those users often resort to cloud training services (aka *Training-as-a-Service*), seeking to obtain an accurate model at a modest cost. To that end, properly setting up the learning rate and batch size is crucial. While various Hyper-parameter Optimization (HPO) methods have been proposed in that regard, they largely act based on heavy-weight validation signals, being inefficient in the overall cost. We find that the model training process can be viewed as a two-dimensional voting process—with gradients for different iterations and from different samples; moreover, to attain cost-efficient training is to ensure that the gradient redundancy is within a proper range—which we find is generic across diverse models. Based on that insight, we introduce GR-Gauge, a general method that gauges the gradient redundancy to instruct HPO decisions like configuration searching and trial termination. Moreover, to improve overall training efficiency in shared clusters, we extend GR-Gauge to be resource-aware, by choosing the most appropriate batch size allowed within the healthy range. Extensive experiments demonstrate that GR-Gauge can help attain near-optimal accuracy, with a time reduction of over 80%; meanwhile, its resource-aware extension reduces the average job queuing time by 77% in a shared GPU cluster.**

***Index Terms*—Hyperparameter Optimization, Gradient Redundancy, Resource Efficiency, Training-as-a-Service**

I. Introduction

Artificial intelligence has revolutionized various application service domains like image recognition [36] and text generation [59], [23]. Many non-professional users now step in by developing their own neural network models [3], [6]. Those non-professional users, like amateurs and start-ups without dedicated GPU clusters, are usually cost-sensitive; they often resort to cloud services for model training, i.e., *Training-as-a-Service* (TaaS) [2], [5], [4], expecting to obtain a model with satisfiable accuracy at a modest cost [1]. In particular, due to the lack of expertise, they largely rely on the service providers to help choose learning rate and batch size—two most classical hyper-parameters—for general user models [9], [11]. In the meantime, for the TaaS provider, when serving multiple training jobs concurrently, it needs to attain low average job completion time with minimum job queuing delay.

To properly setup model training hyper-parameters, a typical methodology for efficient TaaS is Hyper-Parameter Optimization (HPO). HPO means to automatically find the optimal training hyper-parameters with a series of trial runs [8], [9], [11]. It essentially involves making two decisions (searching and scheduling): what hyper-parameters to search in the next trial—with methods like grid search [8] or Bayesian Optimization [11], and how many resources to allocate for each trial—with early-termination methods like Successive Halving [25] and Hyperband [33]. Nonetheless, existing HPO methods heavily rely on the validation metrics like accuracy when making such searching and scheduling decisions, which, although general, suffers low efficiency due to high validation overhead and low decision quality, not appropriate for TaaS adoption.

In fact, when optimizing learning rate and batch size for efficient TaaS, we can leverage the runtime gradient statistics to attain higher efficiency. Specifically, we notice that the model training process can be viewed as a *two-dimensional voting process*: in the *temporal* dimension, the (global) gradients in different iterations continuously vote for the parameter updating direction; in the *spatial* dimension, the (local) gradients from different samples vote for the overall parameter updating direction in each iteration. To make such voting process cost-efficient, we need to avoid trivially consistent or counteracting voting. That is, we need to ensure that the training process in TaaS is in a *healthy* state—with a modest gradient redundancy level. In that regard, we propose two metrics, *temporal gradient redundancy* and *spatial gradient redundancy* (GR_T and GR_S), to help quantify the health level of a model training process. A series of testbed measurements further show that, the early-stage values of GR_T and GR_S can indeed reflect the cost-efficiency (i.e., health) of the entire model training process; meanwhile, their ranges indicating healthy training are *stable across diverse models*. Such a healthy range can be leveraged as a general gauge to guide the HPO process in TaaS.

In this paper, we propose GR-Gauge, a generic method that attains cost-efficient TaaS by gauging the gradient redundancy. Our theoretical analysis shows that there exists a clear correlation between (GR_T, GR_S) and (learning rate, batch size); with the healthy (target) range of (GR_T, GR_S), we accelerate the HPO process in TaaS with three measures. First, we can attain efficient early-stopping by periodically checking whether the GR_T and GR_S values are within the healthy range. Second, based on the gap between the measured (GR_T, GR_S) values and the targeted values indicating healthy training, we can

Guanjie Wang, Chen Chen, Yifei Liu, Quan Chen and Minyi Guo are all with the School of Computer Science, Shanghai Jiao Tong University (email: irfnfnkemed@sjtu.edu.cn, chen-chen@sjtu.edu.cn, nephrencake@sjtu.edu.cn, chen-quan@cs.sjtu.edu.cn and guo-my@cs.sjtu.edu.cn).

Qiang Wang is with the Department of Computer Science and Technology, Harbin Institute of Technology (Shenzhen) (email: qiang.wang@hit.edu.cn).

Cong Wang is with the Department of Computer Science, City University of Hong Kong (email: congwang@cityu.edu.hk).

A preliminary version of this article, titled "GR-Gauge: Cost-efficient Training Configuration By Gauging the Gradient Redundancy" [60], was published in CVPR 2026.

determine the next configuration to explore. Third, we allow the intermediate state of an aborted trial be reused by the next trial—if the (GR_T, GR_S) value can timely return to the healthy range after training resumes; this can reduce resource wastage consumed by trial failures.

Beyond single-job HPO, GR-Gauge can also be extended to resource-aware HPO in shared TaaS platforms. The key observation is that the targeted spatial gradient redundancy (GR_S) provides a controllable knob for balancing between per-job efficiency and cluster-level overall efficiency (i.e., average job completion time). When cluster resources are abundant, GR-Gauge can target a lower GR_S within the healthy range to select a larger global batch size and improve per-job efficiency. When the cluster is highly contended, it can instead target a higher GR_S within the healthy range to select a smaller batch size—so as to reduce per-job GPU demand and admit more jobs into execution. Based on this observation, we design resource-aware HPO that dynamically adjusts the target GR_S according to runtime resource pressure. This policy preserves fast per-job execution under abundant resources, while improving GPU utilization and reducing queuing delay under intense resource contention, thus highly appealing for TaaS providers.

We evaluate the performance of GR-Gauge by comparing it with popular HPO methods over diverse models. End-to-end measurements show that it can reduce the total time to reach a near-optimal model performance by over 80%. Ablation studies further confirm that it is indeed necessary to apply the health-based guidance to each of three aspects (i.e., configuration-search, early-termination and state-reuse). We further evaluate the resource-adequacy-aware extension through real-cluster experiments and trace-driven simulations. The results show that dynamically adjusting the target GR_S according to cluster pressure reduces queuing delay and improves overall training efficiency under intense resource contention. Sensitivity analyses further show that GR-Gauge remains robust across different design choices and parameter settings. Moreover, the additional overheads brought by enforcing GR-Gauge is much less than the performance benefits.

In summary, this paper makes the following contributions:

- We propose temporal and spatial gradient redundancy, GR_T and GR_S, as lightweight indicators of training health for Training-as-a-Service providers.
- We design GR-Gauge, a gradient-redundancy-guided HPO framework that accelerates configuration search through early termination, guided exploration, and state reuse.
- We extend GR-Gauge to resource-aware HPO for shared training services, by dynamically adjusting the target GR_S according to runtime cluster pressure.
- We conduct extensive evaluations to demonstrate the effectiveness of GR-Gauge, with both testbed measurements and trace-driven simulations.

II. Background

A. Automatic Training Configuration in TaaS

Popularity of training-as-a-service. Due to the high cost to maintain dedicated GPU servers, cost-sensitive users with model training demands often resort to public cloud platforms that provision *Training-as-a-Service* (TaaS) [2], [5], [4]. To use such services, a tenant submits its own model, dataset and validation metric to the service provider—with all the remaining issues (like hyper-parameter configuration, resource provisioning) handled by the service provider [1]. In such scenarios, the users expect their models to promptly reach a high accuracy at a low cost, and the provider expects to have a high resource utilization when serving many users concurrently.

Hyper-parameter configuration is critical to TaaS. Facing diverse model training tasks, the TaaS provider needs to automatically configure the training process in a user-transparent manner—in this paper we focus on *learning rate* and *batch size*, which are the most significant hyper-parameters in model training. In the literature, it has been shown that setting them improperly (either over-large or over-small) would lead to poor training performance (in terms of *time-to-target-accuracy*) [34], [16], [56]. Yet, it remains a challenging task for the service provider to properly set up the learning rate and batch size for arbitrary models. Specifically, we focus on promptly finding a proper (learning rate, batch size) setup within the early training period. While some online-tuning methods [41], [12], [52], [27] have proposed to gradually scale the learning rate or batch size in an online manner during the entire training process, their effectiveness still depends on the initial hyper-parameter setup. In fact, existing works [66], [65] have identified the existence of the *critical learning period* in model training: if the initial hyperparameter setup is improper and not remediated timely, the ultimate model training performance would be permanently damaged (regardless of how the hyper-parameters are tuned later). Therefore, to realize cost-efficient TaaS, it is in critical need to—for general models—properly initialize the learning rate and batch size.

Hyper-parameter configuration also affects overall cluster performance. Beyond the efficiency of an individual training job, shared TaaS platforms also need to consider cluster-level resource contention, which has been shown to significantly affect queuing delay and GPU utilization in multi-tenant GPU clusters [26]. The selected batch size affects not only the convergence behavior of a job, but also its GPU demand during execution. A larger batch size may improve per-job execution efficiency once the job starts, but it often requires more GPUs and can increase the queuing delay of other jobs under congestion. Conversely, a smaller batch size can reduce per-job GPU demand and allow more jobs to run concurrently, at the cost of slightly longer per-job execution. Therefore, in shared TaaS scenarios, automatic training configuration should not only identify a healthy learning-rate and batch-size setup, but also adapt such decisions to the runtime resource adequacy of the cluster [52].

B. Related Works

Job-level hyper-parameter optimization. In the literature, Hyper-Parameter Optimization (HPO) [9], [11] is often adopted to automate hyper-parameter configurations. HPO algorithms typically involve two key parts: *searching*—which selects the next hyperparameter candidate to evaluate, and *scheduling*—

which determines how many resources are invested for evaluating each candidate. Regarding the *searching* part, typical algorithms include *uninformed searching* (such as grid search and random search) [8] and *Bayesian Optimization* (BO) [55], [11]. BO uses a surrogate model (e.g., Gaussian process [57], [17]) to guide the selection of candidate configurations. Regarding the *scheduling* part, early stopping mechanisms used in multi-fidelity schemes—including Successive Halving (SH) [25] and Hyperband (HB) [33], choose to early-terminate poorly performing configurations (based on the validation metrics like accuracy) to save resources for more promising candidates. Further, BOHB [15] combines BO and HB to harvest the benefits in both sides—fast candidate identification (from BO) and light-weight per-candidate exploration (from HB).

In particular, existing HPO methods prevalently rely on *external validation metrics* (e.g., accuracy) to make scheduling and searching decisions, so as to be generally applicable to diverse hyper-parameters (e.g., system hyper-parameters like learning rate, and model architecture hyper-parameters like layer number). However, for TaaS scenarios, using such validation metrics for HPO is deficient in three aspects.

- First, with large validation datasets, calculating the validation metric itself would be quite time-consuming. For example, when training a BERT model on an RTX 2080 Ti GPUs, it takes around 250 seconds to conduct a single run over the validation dataset to calculate the F1 score.
- Second, the validation metrics may be inaccurate for making scheduling decisions of HPO (e.g., identify improper setups for early-termination). While existing HPO methods assume good shape of the validation metric curve in training [25], some recent works [51], [40] have found that such an assumption may not hold in practice—good hyper-parameters may also perform badly at the early training stages.
- Third, external validation metrics are not able to guide efficient hyper-parameter searching. While a low accuracy indicates an improper configuration, it fails to suggest which subsequent searching direction is more promising.

In fact, when specifically optimizing learning rate and batch size, we can trade hyper-parameter generality for HPO efficiency, by leveraging the runtime training signals like gradient statistics. Gradient-based signals in the model training process are always readily-observable for the TaaS service provider. In this paper, we show that such gradient-based information—by suggesting a precise exploration direction and timely stopping a diseased trial—can help attain efficient HPO specifically for learning rate and batch size.

Cluster-level resource scheduling. In shared TaaS platforms, jobs can suffer from queuing delay when the overall resource is under intense contention, and it is also crucial to properly allocate the available GPU resources to pending training jobs [52]. Therefore, apart from single-job HPO, many DL cluster schedulers have been proposed to improve job completion time, GPU utilization, fairness, and heterogeneous resource allocation [19], [64], [43], [49]. However, these schedulers assume that the initial resource demand of a job (correlated to the initial batch size setup) must be explicitly specified by users, lacking the flexibility to make adjustment from the TaaS provider's point of view. Consequently, they fail to adapt well to the instantaneous resource contention levels, and are suboptimal in overall job performance (i.e., average job completion time inflated due to long queuing delay, as can be confirmed later in Sec. VI-D). Recent systems further connect training dynamics with resource allocation, for example by co-optimizing statistical efficiency and system throughput or reallocating resources among HPO trials [52], [37]. Yet they make such adaptation inside the scheduler: Pollux, for example, periodically re-allocates GPUs and re-tunes the batch sizes of running jobs, which requires replacing the cluster scheduler and resizing jobs at runtime. In contrast, our resource-aware HPO adapts the resource demand in HPO and leaves job admission and placement to the cluster scheduler; the two approaches are thus complementary. Due to space limitation, we discuss further related work in Appendix H.

TABLE I: Models and datasets used in our experiments.

Task	Model	Dataset	Validation
Recommendation	NeuMF [22]	MovieLens [20]	Hit Rate
Image Classification	ResNet18 [21]	Cifar10 [30]	Accuracy
Question Answering	Bert [13]	SQuAD [53]	F1 Score
Text to image	Stable-diffusion-v1-5 [54]	Naruto BLIP captions [10]	Loss
Instruction Fine-tuning	Llama-3.2-3B-Instruct [46]	FineTome-100k [48]	Loss

III. Motivation

For resource-efficient TaaS, we first explore how gradient statistics can be exploited to judge the true model training efficiency is under the current hyper-parameter setup.

A. The Metric of Gradient Redundancy

Model training: a two-dimensional voting process. Given a machine learning model, the objective of training is to find the optimal parameters that minimize the loss function, which can be expressed as $\omega_{t+1} = \omega_t - \gamma_t g_t$, where ω_t are the parameters at iteration t, g_t is the gradient of loss function (calculated from a set of samples), and γ_t is the learning rate. We note that the model training process can be viewed as conducting two-dimension gradient voting.

First, from a *temporal* perspective, the training process can be viewed as the accumulation of gradient updates over iterations. The final parameters ω can be expressed as $\omega = \omega_0 - \sum_t \gamma_t g_t$. Each iteration t contributes a gradient g_t, and the training process aggregates these contributions over time. In this perspective, each iteration "votes" its gradient to guide the optimization. Second, from a *spatial* perspective, the training process involves aggregating the local gradients from multiple samples (organized in mini-batches) within a single iteration. The parameter update at iteration t can be expressed as $\omega_{t+1} = \omega_t - \frac{1}{n}\gamma_t \sum_{i=1}^{n} g_{t_i}$, where each sample i computes a local gradient g_{t_i}, and the applied update is the average of these gradients. In this perspective, each sample "votes" its local gradient.

Gradient voting redundancy: an effective angle to percept training efficiency. We now think out of the box. With the

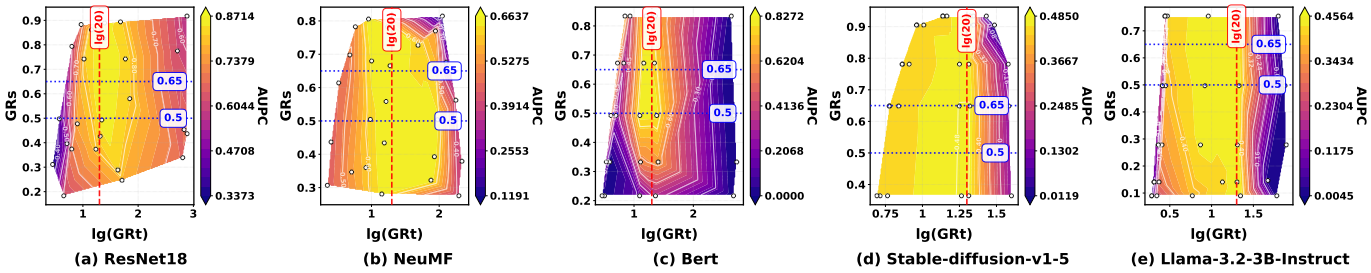


Fig. 1: The relationship between (GR_T, GR_S) and the ultimate AUPC performance for all models described in Table I.

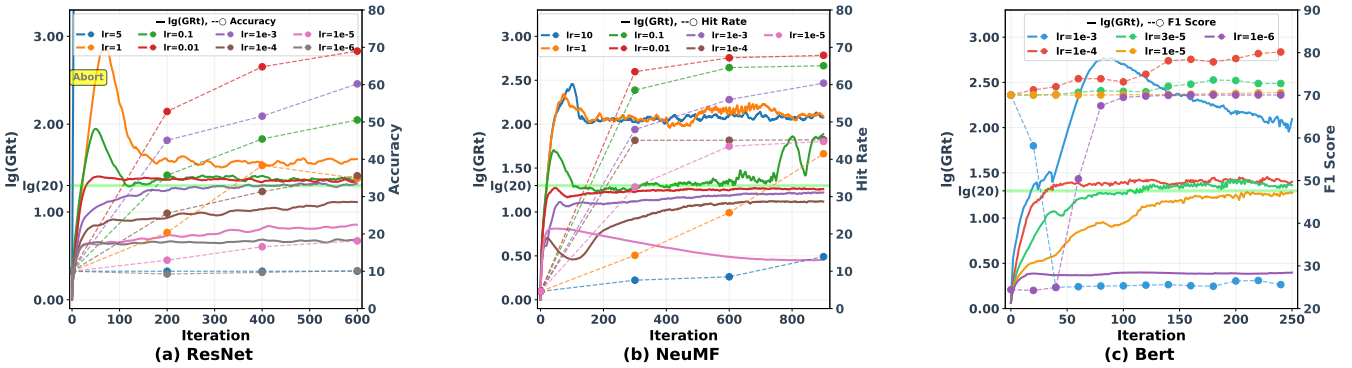


Fig. 2: GR_T and validation metrics across iterations under different learning rate configurations in critical period.

gradient-voting point of view, we note that the efficiency of training depends on the redundancy of 'votes' (i.e., gradient) among different voters. On the one hand, when the gradients from different voters demonstrate significant redundancy, it implies that the training process is reiterating the same information, resulting in inefficient use of computational resources. On the other hand, if the gradients are highly inconsistent, it leads to parameter oscillations caused by conflicting updates, also resulting in a significant reduction in training efficiency. Both training situations can thus be deemed "unhealthy", suffering compromised training efficiency (in terms of the accuracy enhancement from per processed sample). Therefore, to ensure efficient training is to maintain a *proper degree* of gradient redundancy in both dimensions (to be confirmed later in Fig. 1).

Yet, it remains unclear what specific level of gradient redundancy is "proper". To find out, we first propose two metrics that respectively quantifies the gradient redundancy level in the temporal and spatial dimension.

Quantitative metric for depicting temporal and spatial gradient redundancy. In quantifying the gradient redundancy, we maintain two separate metrics for the two orthogonal voting dimensions: GR_T—the temporal-dimension metric which depicts the consistency level among the global gradients from different iterations, and GR_S—the spatial-dimension metric which depicts the consistency level among the local gradients from different samples. To be specific, motivated by gradient-based adaptive optimizers like AdaGrad [14] and Adam [29], we have the following definitions.

Definition 1 (Temporal Gradient Redundancy)**.** *The temporal gradient redundancy metric GR_T of the training at iteration t is defined as*

$$GR_T(t) = \frac{d \cdot \hat{v}_t}{\hat{m}_t{}^2}, \tag{1}$$

where d is the dimension of the g_t, and $\hat{m}_t$ and $\hat{v}_t$ respectively represent[1] *the bias-corrected first and second moment estimation (β is constant smooth factor, which is defaulted to 0.9 in this paper). That is, $\hat{m}_t = m_t/(1-\beta^t)$ and $\hat{v}_t = v_t/(1-\beta^t)$, where $m_t = (1-\beta)\cdot g_{t-1} + \beta \cdot m_{t-1}$ and $v_t = (1-\beta)\cdot g_{t-1}^2 + \beta \cdot v_{t-1}$.*

Definition 2 (Spatial Gradient Redundancy)**.** *Let g_{t_i} be the sampled gradient of sample i at iteration t, then the spatial gradient redundancy metric GR_S of the training at iteration t is defined as:*

$$GR_S(t) = \frac{||\sum_{i=1}^{B} g_{t_i}^2||_1}{||\sum_{i=1}^{B} g_{t_i}||_2^2}, \tag{2}$$

where B is the total number of samples processed in an iteration (i.e., global batch size).

B. A Model-generic Health Criterion

Given the above two metrics, we then show that 1) *the (GR_T, GR_S) value during the critical period is a crucial indicator of the overall training efficiency*, and 2) *the ideal metric values that indicate healthy training are stable across diverse models.*

As shown in Table I, we select five representative models, and train them under different (learning rate, batch size) configurations; each model is trained for a fixed number of

[1] $\hat{m}_t$ and $\hat{v}_t$ are coordinate-wise; we first calculate the redundancy metric for each coordinate and then calculate the global average.

epochs (20, 60, 2, 3 and 10, respectively, which are sufficient for convergence). In particular, to uniformly describe the overall training efficiency for diverse models (which may have a non-accuracy performance criterion like hit-rate or loss), we devise a metric called *Area-Under-Performance-Curve* (AUPC). AUPC shows the average normalized performance[2] attained during the model training process. A higher AUPC value suggests that the model can attain a better performance with less computing cost, thus preferred by general users. For each model, we conduct a series of measurements under different (learning rate, batch size) setups, and Fig. 1 depicts the relationship between the (GR_T, GR_S) status (at iteration 50, an early training stage) and the ultimate AUPC value. In Fig. 1, for each model there exists a *healthy zone* for (GR_T, GR_S) which can yield a good training performance—this aligns with our previous claim that both an over-large and over-small gradient redundancy would be inefficient.

Moreover, we can observe that the healthy zones across different models are similar: *the ideal values of GR_T are mostly all close to 20, and meanwhile the ideal values of GR_S are mostly within the range of [0.5, 0.65] (we use the average, 0.58, as the default target)*. In fact, such a common healthy range is not a coincidence but endorsed by our derivative analysis and the industrial practices of OpenAI [44], which will be detailed in Appendix E. In particular, the GR_S range is defined relative to the gradient noise of the training data [44]; a model or dataset with noisier gradients thus shifts the batch size that attains the range, rather than the range itself. This generality is further confirmed in the temporal dimension by Fig. 2. During the critical learning period, a well-configured run continuously stabilizes around the target GR_T value, whereas misconfigured runs persistently deviate from it and exhibit degraded validation performance. This suggests that the healthy target remains informative throughout the critical period, rather than being valid only at a single early-stage snapshot.

Based on the above analysis, we learn that the gradient redundancy metrics can play as a generic gauge on the health (efficiency) of a given hyper-parameter configuration. Next we show how to leverage such a gradient-redundancy gauge (GR-Gauge) to attain efficient HPO.

IV. Efficient HPO with GR-Gauge

In this section, we propose GR-Gauge, a general method that helps to efficiently set up the learning rate and batch size in training by gauging the gradient redundancy.

A. Correlation Analysis

While GR_T and GR_S can help judge whether a training state is healthy or not, it is still unclear how to approach the healthy state, because GR_T and GR_S themselves cannot be directly adjusted. To that end, we mathematically derive the relationship between (GR_T, GR_S) and (learning rate, batch size). Based on some common assumptions (elaborated in Appendix A), we first have the following two lemmas:

Lemma 1. *Denote the learning rate by η and the average eigenvalue of the Hessian matrix H of the loss function by $\bar{\lambda}$. Denote the global batch size by B and the gradient noise scale by μ. β is the constant smooth factor defined in Definition 1. The expected stable value of GR_T, $\mathbb{E}[GR_T]$, is around*

$$\frac{\left((1+\frac{\mu}{B})^2(1-\bar{\lambda}\eta-\beta)^2\right)\left(\beta^{\frac{1}{1-\beta}}-\left(1-\frac{2\bar{\lambda}\eta}{1+\frac{\mu}{B}}\right)^{\frac{1}{1-\beta}}\right)}{(1-\beta)\left(2\bar{\lambda}\eta-(1-\beta)\left(1+\frac{\mu}{B}\right)\right)\left(\beta^{\frac{1}{1-\beta}}-\left(1-\bar{\lambda}\eta\right)^{\frac{1}{1-\beta}}\right)^2}. \tag{3}$$

Lemma 2. *Under the Assumption 1, Assumption 2 and Assumption 3, denote the global batch size by B and the gradient noise scale by μ. The theoretical expected stable value of GR_S is*

$$\mathbb{E}[GR_S] \approx \frac{1+\mu}{B+\mu}. \tag{4}$$

The proofs of the two Lemmas can be found in Appendix B and Appendix C, respectively.

Based on the above Lemmas, we further derive that there exists a monotonous correlation between (GR_T, GR_S) and (learning rate, batch size), expressed by the following theorems:

Theorem 1. *There exists a positive partial derivative relationship between the learning rate η and the temporal gradient redundancy GR_T:*

$$\frac{\partial}{\partial \eta}\mathbb{E}[GR_T] > 0. \tag{5}$$

Theorem 2. *There exists a negative partial derivative relationship between the batch size B and the spatial gradient redundancy GR_S:*

$$\frac{\partial}{\partial B}\mathbb{E}[GR_S] < 0. \tag{6}$$

The proofs of Theorem 1 and Theorem 2 can also be found in Appendix B and Appendix C. We further empirically examine these relationships across all models in Table I. As shown in Fig. 3, the first row consistently shows an increasing trend of GR_T with the learning rate at a fixed batch size, while the second row shows a decreasing trend of GR_S with the batch size at a fixed learning rate. Although the absolute metric values and surface shapes vary across models, the correlation directions remain stable. This model-consistent monotonicity provides an actionable signal for determining the adjustment directions of the learning rate and batch size. Next, we show how to leverage such correlations to optimize HPO efficiency with the guidance of GR_T and GR_S.

B. (GR_T, GR_S)-guided HPO

Based on the target healthy range of (GR_T, GR_S) and the above correlation, we can act more wisely in HPO. Specifically, we note that gradient redundancy can help optimize HPO efficiency in three aspects. First, they can tell when to stop an ongoing trial. Second, they can suggest a more promising candidate configuration to search next. Third, they can dictate

[2]Some models may have non-accuracy performance metrics like loss or F1 score [53]. In each case, we map the initial metric value to 0 and map the best metric value (best one ever appeared under any setup) to 100%; the instantaneous training performance is normalized by such a range.

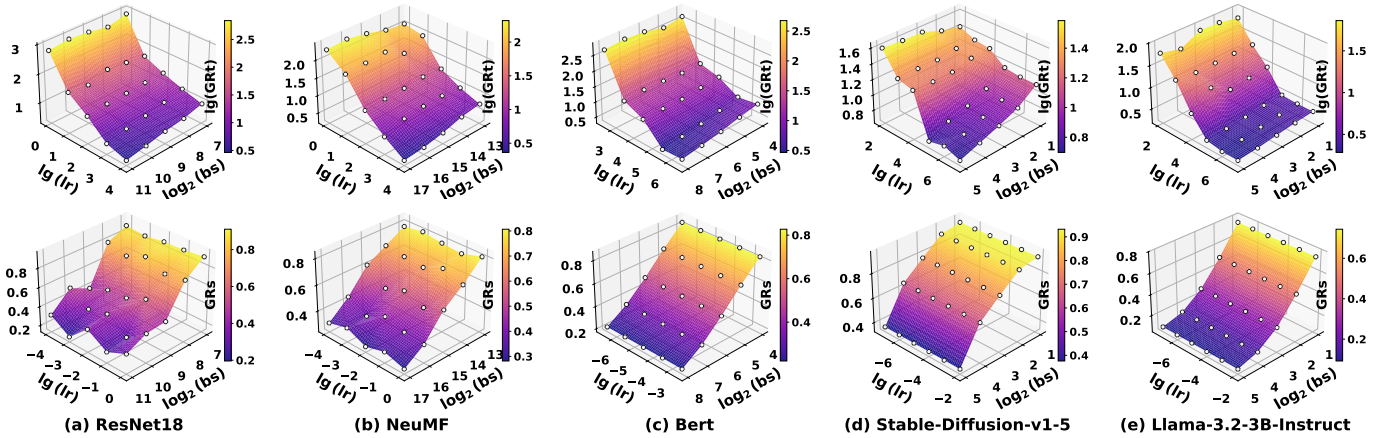
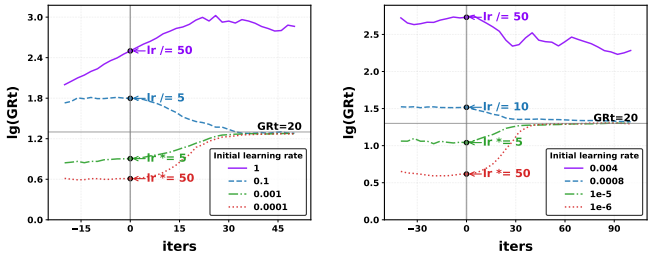
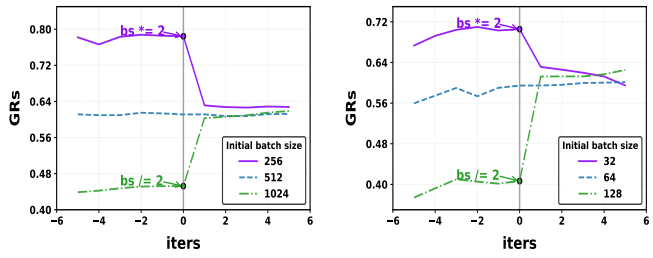


Fig. 3: Explore the expected GR_T and GR_S values (at iteration-50 which represents the early training stage) with different (learning rate, batch size) settings, for all models described in Table I. **Row 1**: GR_T metric value. **Row 2**: GR_S metric value.



(a) ResNet (b) Bert

Fig. 4: Trajectory of GR_T if reusing the aborted state when switching to a new learning rate setup.



(a) ResNet (b) Bert

Fig. 5: Trajectory of GR_S if reusing the aborted state when switching to a new batch size setup.

whether the aborted training state can be reused by the next trial—an optimization angle rarely explored before.

Optimization-1: GR-guided scheduling (early-stopping). We first attain efficient early-stopping by periodically checking GR_T and GR_S. Specifically, we keep[3] checking GR_T and GR_S once after every k (defaulted[4] to 5) iterations; if their values are out of the healthy range, the current trial can be viewed as failed and we need to move on to the next trial. This way, we can timely identify improper configurations. Note that we skip the first 20 iterations in each new trial to confront gradient fluctuations at training commencement.

Optimization-2: GR-guided searching. We then estimate the next configuration to search in HPO based on the gap between the current value of (GR_T, GR_S) and the healthy target values $(GR_T^*, GR_S^*) = (20, 0.58)$. We adjust the learning rate upward when GR_T is below its target and vice versa; similarly, we adjust the batch size downward when GR_S exceeds its target value and vice versa. As shown in Algorithm 1, in practice we scale up the learning rate by $(GR_T^*/GR_T)^{1/\xi_T}$, where ξ_T controls the aggressiveness of adjustment (defaulted to 3); we scale up the batch size by $(GR_S/GR_S^*)^{1/\xi_S}$, where ξ_S controls the aggressiveness of adjustment (defaulted to 1).

Optimization-3: GR-guided state-reusing. Additionally, we note that in many cases the next trial in HPO can reuse the intermediate training states in previous trials. As shown in Fig. 4 and Fig. 5, if we switch to another configuration without resetting the training states, in many cases (but not always) the (GR_T,GR_S) values can still be remediated to the healthy state, and we further find the ultimate training accuracy is still similar to that with restarted training (as endorsed by our later evaluation). Therefore, when switching to another configuration, we can try inherit the training state in the aborted trial (i.e., continue training)—by checking whether the GR_T and GR_S metric value can be restored to the healthy state after an exemption period (defaulted to 20 iterations). If not—suggesting that the model has been irreversibly damaged by improper setup—we still restart training from scratch. This way, we can avoid wasting the previous training efforts and make HPO more efficient. Furthermore, while state reuse may slightly affect the final model quality, we show in Sec. VI-C that the resulting gap is minor and acceptable in practice.

[3]We only check training health in the critical period—identified with the same gradient-norm based method as in [66], where the detection threshold is set to 0.01, and the typical critical period takes up around 10% of the total iterations. Critical-period is sufficient for shaping the training process into a good mode, and classical optimizers like AdamW [41] can usually work well in handling the remaining phases.

[4]While several hyper-parameters like k are newly-induced by GR-Gauge, it is much less model-sensitive. As shown later in our evaluations (Fig. 11), the default hyper-parameters of GR-Gauge we identified can generally work well across diverse models, which are different with the learning rate and batch size hyper-parameters that should be made model-specific.

Algorithm 1: (GR_T, GR_S)-guided HPO

Input : $i, lr, bs, GR_T^*, GR_S^*$ ▷ i: current iteration; lr: learning rate; bs: batch size; $GR_T^* = 20$, $GR_S^* = 0.58$: target metric values; default hyperparameters $k = 5, T = 20, \xi_T = 3, \xi_S = 1$

if $i \pmod k = 0$ **and** $i > T$ **then**
  calculate GR_T, GR_S based on Eq. 1 and Eq. 2
  if $|\frac{GR_T - GR_T^*}{GR_T^*}| > 0.1$ **then**
    $lr \leftarrow lr \times (GR_T^*/GR_T)^{1/\xi_T}$ ▷ If GR_T falls around the expected value, do nothing; else perform reconfiguration.
  if $|\frac{GR_S - GR_S^*}{GR_S^*}| > 0.1$ **then**
    $bs \leftarrow bs \times (GR_S/GR_S^*)^{1/\xi_S}$ ▷ If GR_S falls around the expected value, do nothing; else perform reconfiguration.
  if *GR_T, GR_S not return to healthy state* **then**
    restart training with new lr and bs

V. Resource-Aware HPO

For a TaaS provider, the overall resource demand from public users often fluctuates over time, e.g., following a *diurnal* pattern [62]. In such a shared environment, the HPO decision for one job affects not only its own training efficiency, but also the resource availability of other jobs. Specifically, when the cluster is lightly loaded, assigning more GPUs to a job can shorten its execution time; however, when many jobs compete for limited GPUs, a large per-job GPU demand may delay job admission, aggravate resource fragmentation and increase the queuing delay of pending jobs. In such scenarios, to improve the overall training efficiency of the entire cluster, it is necessary to make HPO actions adaptive to the instantaneous resource contention intensity.

In this section, based on GR-Gauge, we further propose resource-aware HPO. We note that the target value of GR_S provides a natural knob for realizing such adaptation. Recall that in Sec. III-B, we derived that the healthy range of GR_S is $[0.5, 0.65]$, and any target within this interval can yield a satisfactory job-level training performance. Meanwhile, according to Theorem 2, GR_S is negatively correlated with the global batch size. Therefore, the lower bound, $GR_S = 0.5$, corresponds to the largest allowable global batch size, whereas the upper bound, $GR_S = 0.65$, corresponds to the smallest one. In our implementation, the local batch size on each GPU is fixed, so changing the global batch size also changes the number of GPUs demanded by a job. At cluster level, such elasticity of the healthy GR_S range presents a promising opportunity to improve the overall efficiency of multiple jobs contending for the cluster resources.

We first examine the resource elasticity provided by these two endpoints. For each model, we use the configuration corresponding to $GR_S = 0.65$ as the *small-batch configuration* and that corresponding to $GR_S = 0.5$ as the *large-batch configuration*. As shown in Fig. 6, the two configurations achieve comparable final validation performance under the same

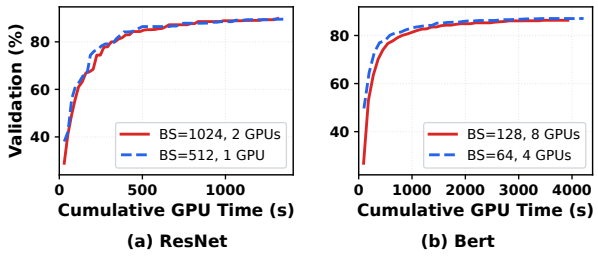


Fig. 6: Validation performance versus cumulative GPU time under the batch sizes corresponding to $GR_S = 0.65$ and $GR_S = 0.5$.

GPU time budget. That is, the two endpoints provide distinct resource operating points without noticeably affecting the attainable model quality. That said, regarding the instantaneous resource (GPU-number) demand, the small-batch configuration substantially reduces the number of GPUs required by each job, enabling a job to utilize fragmented GPU capacity and also mitigating the job backlogging level.

Nevertheless, neither endpoint is universally preferable. Although the small-batch configuration requires fewer GPUs, it may increase the wall-clock execution time of an individual job; by contrast, the large-batch configuration can use more GPUs to finish an admitted job more promptly in cases where the resources are abundant. The batch-size choice therefore involves a trade-off between per-job execution efficiency and cluster-level resource utilization. When resources are abundant, the large-batch endpoint is preferable for shortening job execution; when resources are deficient, however, the small-batch endpoint is preferable because it can lower the resource requirement for admission, improve placement flexibility and enable better utilization of fragmented GPU resources, thereby helping to reduce the average job completion time. This trade-off motivates our design of *resource-aware HPO*, which dynamically switches between the two GR_S points based on the runtime cluster resource contention level.

Under resource-aware HPO, there are two key steps: first to judge whether the cluster resource contention is intense, and second, based on that judgment, to adjust the targeted GR_S value. Specifically, to quantify the resource contention level, we define the *resource pressure* metric, P, as

$$P = \frac{N_{\text{run}} + \beta \sum_{j \in \mathcal{Q}} \hat{N}_j}{G},$$

where G denotes the total number of GPUs in the cluster, N_{run} is the number of GPUs currently occupied by running jobs, $\mathcal{Q}$ is the set of pending jobs, and $\hat{N}_j$ is the estimated GPU demand of pending job j under its current GR_S target. The first term captures the instantaneous GPU occupancy, while the second term estimates the near-future contention introduced by queued jobs. The coefficient $\beta \in [0, 1]$ discounts pending demand because queued jobs do not consume GPUs immediately, but they still indicate upcoming resource pressure (by default, we set $\beta = 0.3$ and evaluate its sensitivity in Sec. VI-F).

Based on P, to balance between job-level and cluster-level training efficiency and also for system stability, we

TABLE II: Hyper-parameter Search Space

Model	learning rate	batch size
ResNet18	$\{10^k\}_{k=-6}^{0}$	$\{2^k\}_{k=6}^{12}$
NeuMF	$\{10^k\}_{k=-6}^{0}$	$\{2^k\}_{k=11}^{18}$
Bert	$\{a \times 10^k\}_{k=-7}^{-3}, a \in \{2, 5\}$	$\{2^k\}_{k=3}^{6}$
Stable-diffusion	$\{10^k\}_{k=-7}^{-2}$	$\{2^k\}_{k=1}^{5}$
Llama-3.2-3B	$\{10^k\}_{k=-7}^{-1}$	$\{2^k\}_{k=1}^{5}$

adopt a hysteresis rule in adjusting the GR_S target under resource-aware HPO. Specifically, we introduce two knobs: τ_{low} and τ_{high}, which respectively represents the lower and upper pressure thresholds. When $P \geq \tau_{\text{high}}$, resource-aware HPO sets the target to the upper endpoint, $GR_S = 0.65$, which corresponds to the small-batch configuration and helps improve cluster-level overall efficiency; when $P \leq \tau_{\text{low}}$, resource-aware HPO sets the target to the lower endpoint, $GR_S = 0.5$, which corresponds to the large-batch configuration and helps improve job-level progressing efficiency. When $\tau_{\text{low}} < P < \tau_{\text{high}}$, resource-aware HPO retains the previous target to avoid unnecessary oscillation caused by short-term pressure fluctuations (by default, τ_{low} and τ_{high} are respectively set to 0.5 and 1.2). Sec. VI-D and Sec. VI-E evaluate—with both testbed measurements and simulations—the effectiveness of this resource-aware HPO adaptation, and Sec. VI-F further studies its robustness under different settings of β, τ_{low}, and τ_{high}.

VI. Evaluation

A. End-to-End Performance

We conduct experiments in a 16-node cluster with totally 64 NVIDIA GeForce RTX 2080 Ti GPUs. With that cluster we train the models from diverse tasks as introduced in Table I. The hyper-parameter space of learning rate and batch size is listed in Table II. For all the models, we adopt the AdamW optimizer [41]. Besides, we include five baselines[5]: *Random Search* (RS), *Successive halving* (SH), *Hyperband* (HB), BOHB, and *cost-frugal optimization* (CFO) [63]. The former four are elaborated in Sec. II-B, and CFO updates the hyper-parameters by continuously sampling their hyper-gradients that can yield lower validity loss, representing a continuous online-tuning method.

Metrics. To evaluate the cost-efficiency of GR-Gauge, we set multiple validation targets and record the GPU-time (absolute time multiplied by the GPU number) required to reach each target. The targets are set close to the optimal validation metric values ever attained under each model-dataset combination. Besides, we also experiment with fixed GPU-time budget and record the validation performance achieved within each budget level. By setting multiple validation targets (aligned with the same method as in Fig. 1) or budgets, we seek to cover diverse cost-accuracy trade-off levels. To mitigate the randomness, we repeat each experiment for 5 times and present the average performance value.

[5]The former four are elaborated in Sec. II-B; CFO represents a continuous online-tuning method based on hyper-gradients, covering a design point absent in the others. Methods in popular HPO frameworks (e.g., Ray Tune [38]) are incremental variants of these core algorithms—ASHA [32] from SH, PBT [24] from HyperBand, and Optuna [7] from BOHB—and are thus covered by our baseline set.

TABLE III: GPU-Time (normalized by GR-Gauge) required to reach different validation targets by different HPO methods. "N/A" indicates that the target is not reached within a reasonable time budget.

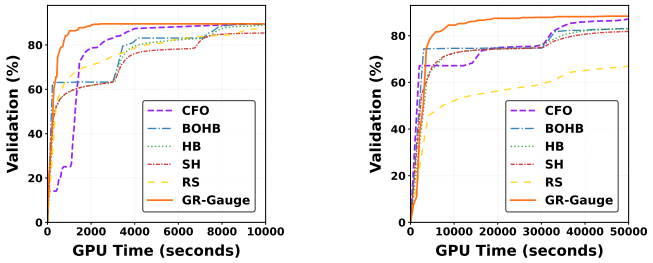
Model	Validation Targets	*RS*	*SH*	*HB*	*BOHB*	*CFO*	*GR-Gauge*
ResNet18	75%	6.42×	7.50×	7.28×	6.64×	3.23×	**504s**
	85%	9.13×	8.56×	7.81×	7.53×	3.54×	**977s**
	89%	6.27×	6.02×	5.47×	4.23×	3.11×	**2010s**
NeuMF	55%	2.44×	0.69×	0.65×	**0.61**×	8.47×	275s
	60%	3.81×	1.17×	1.16×	1.03×	10.6×	**325s**
	68%	N/A	N/A	3.24×	2.75×	N/A	**426s**
Bert	70%	17.8×	2.08×	2.16×	0.82×	4.44×	**3650s**
	80%	24.8×	7.47×	6.96×	6.52×	6.37×	**5090s**
	85%	18.5×	9.27×	7.86×	7.95×	3.32×	**11000s**
Diffusion	85%	1.60×	1.61×	1.53×	1.51×	1.96×	**829s**
	90%	N/A	3.33×	3.44×	3.40×	2.24×	**1640s**
	92%	N/A	3.55×	3.61×	2.50×	2.08×	**2350s**
Llama3	50%	6.26×	2.88×	2.81×	**0.12**×	4.33×	21500s
	55%	3.98×	2.22×	1.83×	1.79×	2.68×	**53400s**
	59%	N/A	2.62×	1.83×	1.30×	2.56×	**75300s**

TABLE IV: Validation performance (in percentage) attained under fixed budgets (GPU-Time) by different HPO methods.

Model	Time Budgets	*RS*	*SH*	*HB*	*BOHB*	*CFO*	*GR-Gauge*
ResNet18	1000s	63.50	58.93	58.08	63.09	25.09	**85.64**
	2000s	69.71	61.74	61.72	63.25	78.58	**88.48**
	4000s	76.95	76.04	79.37	81.03	87.36	**89.51**
NeuMF	200s	50.20	55.76	55.54	**56.94**	41.14	36.18
	500s	54.05	61.15	61.27	61.51	45.22	**68.77**
	1000s	57.80	62.95	66.12	67.53	46.93	**69.11**
Bert	10000s	50.93	71.13	71.34	74.60	67.17	**84.56**
	20000s	55.14	74.15	74.27	74.72	74.45	**87.47**
	40000s	64.33	80.16	82.02	82.28	85.76	**88.19**
Diffusion	1000s	84.41	84.22	82.02	81.62	84.88	**88.60**
	5000s	89.21	89.14	88.98	89.39	92.03	**92.79**
	10000s	89.36	93.76	92.24	93.32	92.19	**94.17**
Llama3	20000s	35.31	41.34	40.04	**52.95**	37.60	42.45
	50000s	39.49	48.97	48.67	53.11	37.60	**54.33**
	100000s	44.97	54.14	55.53	60.54	50.01	**60.81**

Results. Table III shows the results under multiple validation targets. In reaching each validation target, we can see that GR-Gauge requires the shortest training time in most cases. Taking ResNet18 for example, to achieve an accuracy of 85%, GR-Gauge can save the time by 63.6% when compared to BOHB (the second best). Table IV further lists the results under multiple time budgets. It shows that GR-Gauge can make the most prompt enhancement on validation performance: after 20,000s, the F1 score of the Bert model is already 87.47% under GR-Gauge, yet under other methods they are all below 75%. The two experiments can well demonstrate the superiority of GR-Gauge in time-to-accuracy performance.

In Fig. 7, we further depict the instantaneous training performance (i.e., the best validation-metric value ever appeared under any trial of an HPO process) under different HPO methods. It can be observed that GR-Gauge can rapidly attain a high validation performance within a short time. It suggests that a highly cost-sensitive user can attain a satisfiable training performance under GR-Gauge.



(a) ResNet18 (b) Bert

Fig. 7: Curves of the best-ever validation performance under different HPO methods.

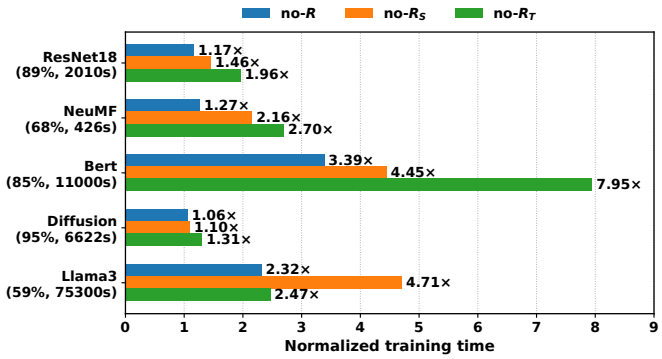


Fig. 8: Training time of GR-Gauge variants normalized to GR-Gauge. GR-Gauge is the $1.0\times$ baseline. Each y-axis label reports the validation target and the absolute training time of GR-Gauge in the format of (target, GR-Gauge time).

B. Ablation Study

Note that GR-Gauge applies health-based guidance in three aspects: state-reuse, configuration-search and early-termination. We now check the necessity of health-based guidance in each aspect. To that end, we introduce three degraded versions of GR-Gauge. In no-*R*, we disable state-reuse; in no-*RS*, we disable both state-reuse and health-guided configuration-searching; in no-*RT*, we disable both state-reuse and health-guided early-termination. Fig. 8 shows the results of the ablation study. It can be seen that all degraded versions require more time than standard GR-Gauge to reach the designated performance target. Meanwhile, no-*RS* and no-*RT* are both less efficient than no-*R*. Therefore, it is indeed necessary to apply health-based guidance in each of the three aspects.

C. Impact of State-Reusing on Final Accuracy

TABLE V: Final validation performance of GR-Gauge and from-scratch training under the best-found configuration.

Method	ResNet	NeuMF	Bert	Diffusion	Llama3
From scratch	90.75	69.86	88.40	95.75	60.54
GR-Gauge	89.51	69.11	88.19	94.17	60.81

Table V reports the final validation performance after the HPO process finishes. We compare GR-Gauge with a *from-scratch run* using the best-found hyperparameter configuration, which serves as a reference for the final model quality without state reuse. Across the evaluated models, the performance gap remains minor, confirming that state-reusing does not cause irreversible damage to model quality. These results show that the efficiency benefit of state reuse does not come at the cost of noticeable final-quality degradation in our evaluated settings. This is consistent with our conditional reuse policy, which restarts from scratch when the inherited state fails to recover to the healthy redundancy range.

D. Testbed Evaluation of Resource-Aware HPO

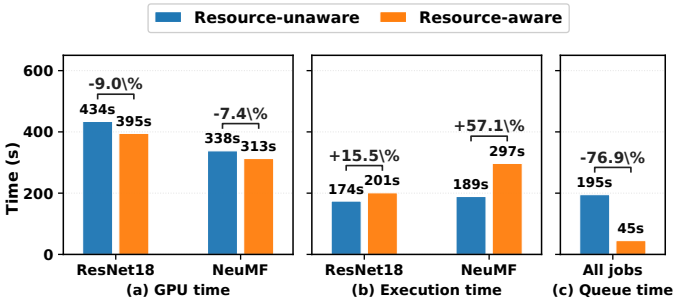


Fig. 9: Real-system effectiveness of resource-aware HPO. Percentages indicate changes from resource-unaware to resource-aware HPO.

Recall that in Sec. V we have introduced resource-aware HPO to balance between job-level training efficiency and cluster-level overall efficiency. We evaluate resource-aware HPO on a realistic cluster with 8 NVIDIA GeForce RTX 3090 GPUs. We submit 10 jobs almost simultaneously, including 5 ResNet18 jobs and 5 NeuMF jobs. Each job terminates after reaching its validation target, i.e., 85% accuracy for ResNet18 and 68% hit rate for NeuMF. This workload creates strong resource contention. In response, GR-Gauge raises the target GR_S toward the upper bound of the healthy range, thereby selecting smaller global batch sizes and reducing the number of GPUs required by each job. Fig. 9 shows the GPU time (execution time multiplied by the number of GPUs), execution time (gap between training start time and training completion time), and queuing time respectively with and without resource-aware HPO. Although using fewer GPUs can slightly increase the average per-job execution time, it in average reduces the per-job GPU time by about 10%. With each job demanding fewer GPUs than before, the job backlogging level can be effectively mitigated: as shown in Fig. 9c, the average queuing time is remarkably reduced from 195s to 45s.

E. Simulated Evaluation of Resource-aware HPO

Apart from testbed measurement, we also evaluate the effectiveness of resource-aware scheduling policy through trace-driven simulations—based on actual training runs on NVIDIA RTX 2080 Ti GPUs. For each task, we profile the execution time and validation performance under two feasible batch-size configurations at the best learning rate. These two configurations, denoted as *small-bs* and *large-bs*, correspond to the upper and lower endpoints of the healthy GR_S range,

TABLE VI: Task settings for simulation.

Model	Validation Target	Small-bs	Large-bs	Max Local BS
ResNet	85% Accuracy	512	1024	512
NeuMF	68% Hit Rate	32768	65536	32768
Bert	85% F1 Score	64	128	16

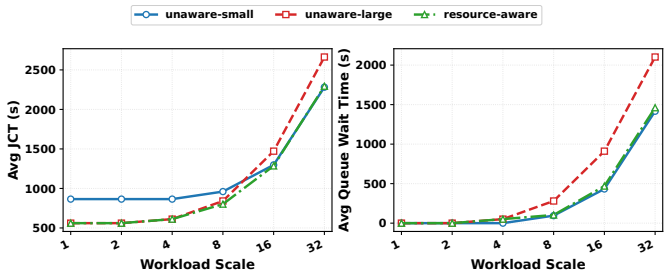


Fig. 10: Average JCT and queue waiting time under increasing workload scales on a fixed 128-GPU cluster.

respectively. The small-bs configuration requires fewer GPUs, while the large-bs configuration uses more GPUs to shorten per-job execution time. To evaluate the policies under different levels of resource contention, we fix the cluster size to 128 GPUs and proportionally scale the submitted workload. The base workload contains $(4, 4, 4)$ jobs for (ResNet, NeuMF, Bert), and the workload at scale s is $s \times (4, 4, 4)$, where $s \in \{1, 2, 4, 8, 16, 32\}$. All jobs are submitted simultaneously and terminate after reaching their corresponding validation targets. We report the average job completion time (JCT). The maximum local batch size of each task is obtained through profiling to ensure that every configuration satisfies the GPU memory constraint. The detailed task and workload settings are summarized in Tables VI. Besides, we compare the following three scheduling strategies: (1) *Unaware-small*: all jobs use the small-bs configuration. This policy lowers the per-job GPU requirement and improves placement flexibility under contention, but may leave GPUs underutilized when only a few jobs are present. (2) *Unaware-large*: all jobs use the large-bs configuration. This policy shortens per-job execution time when sufficient GPUs are available, but its larger admission requirement may increase queuing delay under contention. (3) *Resource-aware*: the proposed policy dynamically selects between the two configurations according to cluster pressure, while keeping the target GR_S within the healthy range.

Fig. 10 shows that *unaware-large* achieves lower JCT at small workload scales, where queuing is limited and larger batches shorten job execution. As the workload increases, its higher GPU demand and poor utilization of fragmented GPU resources cause queuing delay to increase rapidly, while *unaware-small* reduces waiting time at the cost of longer execution. By switching between the two configurations according to cluster pressure, the *resource-aware* method preserves large-batch efficiency under light load and small-batch placement flexibility under contention, allowing each job to operate at a resource point suited to the current cluster condition.

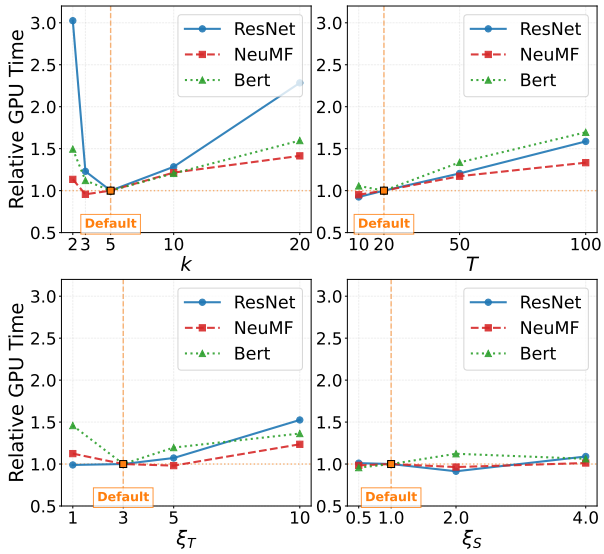
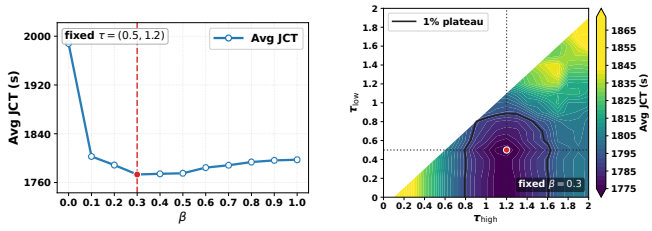


Fig. 11: Relative GPU Time required for models to reach target performance (ResNet18: 85% Accuracy; NeuMF: 60% Hit rate; Bert: 80% F1 Score) under different GR-Gauge hyper-parameters.



(a) Sensitivity to β (b) Sensitivity to $(\tau_{\text{low}}, \tau_{\text{high}})$

Fig. 12: Sensitivity of hyper-parameters for resource-aware HPO.

F. Sensitivity Analysis

Sensitivity of Hyperparameters in standard GR-Gauge. As displayed in Alg. 1, GR-Gauge introduces several hyper-parameters: k, T, ξ_T, ξ_S; here we conduct sensitivity analysis on them. As shown in Fig. 11, we respectively set k, T, ξ_T, ξ_S to different values, and record the resultant GPU time to reach the validation target (normalized by that under our default setup). On the one hand, such GR-Gauge hyper-parameters do affect the overall performance. For example, over-frequent metric checking (e.g., $k = 1$) increases the computation overheads, while insufficient checking (e.g., $k = 50$) hurts the search efficiency. On the other hand, those GR-Gauge hyper-parameters are much less model-sensitive: the default setup we identified works relatively well across different models; GR-Gauge users thus do not need to tune such GR-Gauge hyper-parameters—different from learning rate and batch size which are model-specific.

Sensitivity of Hyperparameters for Resource-aware HPO. As described in Sec. V, β controls how pending jobs are converted into effective resource pressure, whereas $(\tau_{\text{low}}, \tau_{\text{high}})$ determines when resource-aware HPO switches between small-bs and large-bs configurations. By default, we fix $\beta = 0.3$,

TABLE VII: Time breakdown of the GR-Gauge running time.

Model	Validation Targets	GR_T Overhead(s)	GR_S Overhead(s)	Total GPU-Time (s)
ResNet18	89%	56 (2.78%)	28.1 (1.40%)	2013
NeuMF	68%	2.72 (0.63%)	1.11 (0.26%)	426
Bert	85%	216 (1.96%)	346 (3.15%)	11230
Diffusion	95%	159 (2.41%)	220 (3.32%)	6622
Llama3	59%	463 (0.61%)	510 (0.62%)	75320

$\tau_{\text{low}} = 0.5$, and $\tau_{\text{high}} = 1.2$ for all main experiments. Here, we conduct sensitivity analysis on these parameters and report the resulting average JCT in Fig. 12. On the one hand, these parameters do affect scheduling performance. For example, an overly small β underestimates pending demand and may keep jobs in large-bs configurations under contention, leading to longer queuing delay. In contrast, an overly large β overestimates pressure and may make resource-aware HPO switch to small-bs configurations too conservatively, reducing per-job training efficiency. Similarly, inappropriate threshold pairs can either delay switching under contention or trigger overly conservative switching. On the other hand, the performance is not fragile to a single parameter choice. The β curve changes smoothly around the default value, and the $(\tau_{\text{low}}, \tau_{\text{high}})$ heatmap shows a continuous low-JCT plateau around the selected threshold pair. This indicates that the calibrated parameters work well across W1–W3 and different cluster sizes.

G. Overheads

Table VII illustrates the breakdown of GR-Gauge running time, showing the GR_T and GR_S overheads. Compared to the total running time, the overheads to maintain the two metrics are indeed not large (commonly less than 4%). This overhead is modest mainly because of the lightweight design of the measurements and the critical-period monitoring strategy. Specifically, GR_T and GR_S rely on existing gradient information without introducing additional forward or backward passes. Moreover, GR-Gauge measures gradient redundancy *only during the critical period of training* (where early signals are most informative) rather than *continuously* (throughout the entire training process). This design keeps the cumulative monitoring cost small in practice. Compared to the training speedup with GR-Gauge, such overheads are indeed acceptable.

VII. Conclusion

In this paper, we propose GR-Gauge, a novel method that helps set cost-efficient training hyperparameters for Training-as-a-Service providers. We view model training as a gradient-voting process, and find that for efficiency the temporal and spatial gradient redundancy levels be maintained within a healthy range. By comparing the runtime gradient redundancy with the healthy range, GR-Gauge can guide HPO decisions in searching, scheduling and state-reusing. Based on GR-Gauge, we further propose resource-aware HPO, which adapts the initial batch size setup based on the runtime contention level, to improve the overall cluster performance. Extensive experiments demonstrate that GR-Gauge can help achieve cost-effective model training across a diverse range of models and improve cluster-level efficiency across different workload intensities.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (NSFC), under grants 62202300, 62572304 and 62232011. Chen Chen is the corresponding author.

References

[1] Automatic model tuning with sagemaker ai. https://docs.aws.amazon.com/sagemaker/latest/dg/automatic-model-tuning.html, 2025.

[2] Azure machine learning. https://azure.microsoft.com/en-us/products/machine-learning, 2025.

[3] Democratising ai: How startups can train their own large language models. https://medium.com/@PropagenAI/democratising-ai-how-startups-can-train-their-own-large-language-models-ac930b473f28, 2025.

[4] Ibm watson machine learning. https://www.ibm.com/products/watsonx-ai, 2025.

[5] Model training with aws sagemaker. https://docs.aws.amazon.com/sagemaker/latest/dg/train-model.html, 2025.

[6] Small but mighty: The rise of personal ai model training. https://www.linkedin.com/pulse/beginners-guide-building-ai-model-diana-wolf-torres-wxejc/, 2025.

[7] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '19, page 2623–2631, New York, NY, USA, 2019. Association for Computing Machinery.

[8] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *J. Mach. Learn. Res.*, 13(null):281–305, February 2012.

[9] Bernd Bischl, Martin Binder, Michel Lang, Tobias Pielok, Jakob Richter, Stefan Coors, Janek Thomas, Theresa Ullmann, Marc Becker, Anne-Laure Boulesteix, et al. Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 13(2):e1484, 2023.

[10] Eole Cervenka. Naruto blip captions. https://huggingface.co/datasets/lambdalabs/naruto-blip-captions/, 2022.

[11] Hyunghun Cho, Yongjin Kim, Eunjung Lee, Daeyoung Choi, Yongjae Lee, and Wonjong Rhee. Basic enhancement strategies when using bayesian optimization for hyperparameter tuning of deep neural networks. *IEEE access*, 8:52588–52608, 2020.

[12] Aditya Devarakonda, Maxim Naumov, and Michael Garland. Adabatch: Adaptive batch sizes for training deep neural networks. *arXiv preprint arXiv:1712.02029*, 2017.

[13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[14] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011.

[15] Stefan Falkner, Aaron Klein, and Frank Hutter. Bohb: Robust and efficient hyperparameter optimization at scale. In *International conference on machine learning*, pages 1437–1446. PMLR, 2018.

[16] Oleg Filatov, Jan Ebert, Jiangtao Wang, and Stefan Kesselheim. Time transfer: On optimal learning rate and batch size in the infinite data limit. *arXiv preprint arXiv:2410.05838*, 2024.

[17] Eduardo C Garrido-Merchán and Daniel Hernández-Lobato. Dealing with categorical and integer-valued variables in bayesian optimization with gaussian processes. *Neurocomputing*, 380:20–35, 2020.

[18] Joseph Giovanelli, Alexander Tornede, Tanja Tornede, and Marius Lindauer. Interactive hyperparameter optimization in multi-objective problems via preference learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 12172–12180, 2024.

[19] Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. Tiresias: A GPU cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 485–500. USENIX Association, 2019.

[20] F Maxwell Harper and Joseph A Konstan. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*, 5(4):1–19, 2015.

[21] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE CVPR*, 2016.

[22] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *ACM WWW*, 2017.

[23] Touseef Iqbal and Shaima Qureshi. The survey: Text generation models in deep learning. *Journal of King Saud University-Computer and Information Sciences*, 34(6):2515–2528, 2022.

[24] Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M. Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, Chrisantha Fernando, and Koray Kavukcuoglu. Population based training of neural networks, 2017.

[25] Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *Artificial intelligence and statistics*, pages 240–248. PMLR, 2016.

[26] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. Analysis of Large-Scale Multi-Tenant GPU clusters for DNN training workloads. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 947–960. USENIX Association, 2019.

[27] Tyler Johnson, Pulkit Agrawal, Haijie Gu, and Carlos Guestrin. Adascale sgd: A user-friendly algorithm for distributed training. In *International conference on machine learning*, pages 4911–4920. PMLR, 2020.

[28] Arlind Kadra, Maciej Janowski, Martin Wistuba, and Josif Grabocka. Scaling laws for hyperparameter optimization. *Advances in Neural Information Processing Systems*, 36:47527–47553, 2023.

[29] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[30] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

[31] Sunwoo Lee, Tuo Zhang, and A Salman Avestimehr. Layer-wise adaptive model aggregation for scalable federated learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 8491–8499, 2023.

[32] Liam Li, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-tzur, Moritz Hardt, Benjamin Recht, and Ameet Talwalkar. A system for massively parallel hyperparameter tuning. In I. Dhillon, D. Papailiopoulos, and V. Sze, editors, *Proceedings of Machine Learning and Systems*, volume 2, pages 230–246, 2020.

[33] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.

[34] Mu Li, Tong Zhang, Yuqiang Chen, and Alexander J Smola. Efficient mini-batch training for stochastic optimization. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 661–670, 2014.

[35] Yang Li, Yu Shen, Huaijun Jiang, Wentao Zhang, Jixiang Li, Ji Liu, Ce Zhang, and Bin Cui. Hyper-tune: Towards efficient hyper-parameter tuning at scale. *arXiv preprint arXiv:2201.06834*, 2022.

[36] Yinglong Li. Research and application of deep learning in image recognition. In *2022 IEEE 2nd international conference on power, electronics and computer applications (ICPECA)*, pages 994–999. IEEE, 2022.

[37] Richard Liaw, Romil Bhardwaj, Lisa Dunlap, Yitian Zou, Joseph E. Gonzalez, Ion Stoica, and Alexey Tumanov. HyperSched: Dynamic resource reallocation for model development on a deadline. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 61–73. ACM, 2019.

[38] Richard Liaw, Eric Liang, Robert Nishihara, Philipp Moritz, Joseph E. Gonzalez, and Ion Stoica. Tune: A research platform for distributed model selection and training, 2018.

[39] Ming Lin, Pichao Wang, Zhenhong Sun, Hesen Chen, Xiuyu Sun, Qi Qian, Hao Li, and Rong Jin. Zen-nas: A zero-shot nas for high-performance image recognition. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 347–356, 2021.

[40] Jiesong Liu, Feng Zhang, Jiawei Guan, and Xipeng Shen. Uq-guided hyperparameter optimization for iterative learners. *Advances in Neural Information Processing Systems*, 37:386–415, 2024.

[41] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2017.

[42] Na Lv, Zhi Shen, Chen Chen, Zhifeng Jiang, Jiayi Zhang, Quan Chen, and Minyi Guo. Fedca: Efficient federated learning with client autonomy. In *Proceedings of the 53rd International Conference on Parallel Processing*, pages 494–503, 2024.

[43] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. Themis: Fair and efficient GPU cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 289–304. USENIX Association, 2020.

[44] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162*, 2018.

[45] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training, 2018.

[46] Meta. meta-llama/llama-3.2-3b-instruct, 2024. Model License: Meta Llama 3 Community License.

[47] Paul Micaelli and Amos Storkey. Gradient-based Hyperparameter Optimization Over Long Horizons. In *NeurIPS*, 2021.

[48] Mlabonne. mlabonne/finetome-100k, 2024. Dataset License: Apache-2.0.

[49] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-Aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498. USENIX Association, 2020.

[50] Fabian Pedregosa. Hyperparameter optimization with approximate gradient. In *International conference on machine learning*, pages 737–746. PMLR, 2016.

[51] Zhongyi Pei, Zhiyao Cen, Yipeng Huang, Chen Wang, Lin Liu, Philip Yu, Mingsheng Long, and Jianmin Wang. Bttackler: A diagnosis-based framework for efficient deep learning hyperparameter optimization. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2340–2351, 2024.

[52] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R Ganger, and Eric P Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, 2021.

[53] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*, 2016.

[54] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, June 2022.

[55] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P Adams, and Nando De Freitas. Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2015.

[56] SL Smith. Don't decay the learning rate, increase the batch size. *arXiv preprint arXiv:1711.00489*, 2017.

[57] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems*, 25, 2012.

[58] Ilya Sutskever, James Martens, and George Dahl. On the importance of initialization and momentum in deep learning. In *ICML*, 2013.

[59] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

[60] Guanjie Wang and Chen Chen. Gr-gauge: Cost-efficient training configuration by gauging the gradient redundancy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12934–12943, 2026.

[61] Jianyu Wang and Gauri Joshi. Adaptive Communication Strategies to Achieve the Best Error-Runtime Trade-off in Local-Update SGD. In *SysML*, 2019.

[62] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. {MLaaS} in the wild: Workload analysis and scheduling in {Large-Scale} heterogeneous {GPU} clusters. In *19th USENIX symposium on networked systems design and implementation (NSDI 22)*, pages 945–960, 2022.

[63] Qingyun Wu, Chi Wang, and Silu Huang. Frugal optimization for cost-related hyperparameters. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 10347–10354, 2021.

[64] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610. USENIX Association, 2018.

[65] Gang Yan, Hao Wang, and Jian Li. Seizing critical learning periods in federated learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 8788–8796, 2022.

[66] Gang Yan, Hao Wang, Xu Yuan, and Jian Li. Criticalfl: A critical learning periods augmented client selection framework for efficient federated learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2898–2907, 2023.

[67] Peifeng Yu, Jiachen Liu, and Mosharaf Chowdhury. Fluid: Resource-aware hyperparameter tuning engine. *Proceedings of Machine Learning and Systems*, 3:502–516, 2021.

[68] Yi Zhou, Parikshit Ram, Theodoros Salonidis, Nathalie Baracaldo, Horst Samulowitz, and Heiko Ludwig. Flora: Single-shot hyper-parameter optimization for federated learning. *arXiv preprint arXiv:2112.08524*, 2021.



Guanjie Wang received the bachelor's degree in computer science from Shanghai Jiao Tong University in 2026. He is currently working toward the Ph.D. degree at Carnegie Mellon University. His research interests include machine learning systems.



Chen Chen is an Associate Professor in John Hopcroft Center for Computer Science, Shanghai Jiao Tong University. He received the B.Eng. degree from Tsinghua University in 2014, and the PhD degree in 2018 from Department of Computer Science and Engineering, Hong Kong University of Science and Technology. His recent research interests include distributed deep learning, big data systems and networking. He previously worked as a researcher at the Theory Lab, Huawei Hong Kong Research Center.



Yifei Liu received the M.S. degree from Shanghai Jiao Tong University and is currently an LLM infrastructure researcher at Xiaohongshu. Previously, he worked on LLM inference optimization at Huawei Cloud. His research interests include efficient LLM inference systems, cluster scheduling, and distributed caching.



Qiang Wang is currently an Associate Professor in Harbin Institute of Technology (HIT), Shenzhen China. He was a Research Assistant Professor in the Department of Computer Science, Hong Kong Baptist University before he joined HIT. Dr. Wang received his B.Eng. degree from South China University of Technology in 2014, and the Ph.D. degree in Computer Science from Hong Kong Baptist University in 2020. His research interests include general-purpose GPU computing and machine learning systems. He received the best paper award of EuroSys 2025 and is a member of the IEEE.



Cong Wang is a Chair Professor and the Head of Computer Science Department at City University of Hong Kong. His research spans data security and privacy, AI systems and security, and decentralized application security. He is an IEEE Fellow, a HK RGC Research Fellow, and a Founding Member of the Hong Kong Young Academy of Sciences. He has served as the Editor-in-Chief of IEEE Transactions on Dependable and Secure Computing.



Quan Chen is a professor in the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China. His research interests include High-performance computing, Task Scheduling in various architectures, Resource management in Datacenter, Runtime System and Operating System. He got his Ph.D. degree at June 2014 from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China.



Minyi Guo is currently the Zhiyuan Chair professor of the Department of Computer Science and Engineering, Shanghai Jiao Tong University (SJTU), China. Before joining SJTU, Dr. Guo had been a professor of The University of Aizu, Japan. His present research interests include parallel/distributed computing, compiler optimizations, emerging computer architectures, and cloud computing. He has more than 500 publications in major journals and international conferences in these areas. He received 7 best paper awards from international conferences. He is now the Editor-in-Chief of IEEE Transactions on Sustainable Computing, and on the editorial board of IEEE Transactions on Cloud Computing and IEEE Transactions on Computers. Dr. Guo is a Fellow of IEEE and a Fellow of CCF.

APPENDIX A
ASSUMPTIONS FOR THEORETICAL ANALYSIS

To model the metrics and establish the theoretical analysis, we make the following assumptions. Table A1 summarizes the key notations used in the analysis.

TABLE A1: Key notations used in the analysis.

$g_x(\theta)$	The gradient computed on a data point x at the parameter θ
$G_{est}(\theta)$	The sampled batch gradient of the overall loss, i.e., $G_{est}(\theta) = \frac{1}{B}\sum_{i=1}^{B} g_{x_i}(\theta), x_i \sim \rho$
$G(\theta)$	The gradient of the overall loss, i.e., $G(\theta) = \mathbb{E}_{x\sim\rho}[g_x(\theta)]$
H	The constant Hessian matrix of the overall loss function
μ	The constant scale that quantifies the gradient noise variance, i.e., $\mu = Tr(Cov_{x\sim\rho}[g_x(\theta)])/\|\|G(\theta)\|\|_2^2$
β	The constant smooth factor of the exponential moving average used to calculate GR_T, which is fixed to 0.9 in the paper
η	The learning rate
B	The global batch size

Assumption 1. ***(Gradient Distribution)*** *Assume that for any parameter θ, the expectation and variance of stochastic gradient $g_x(\theta)$ computed on a data point x satisfy that:*

$$\mathbb{E}_{x\sim\rho}[g_x(\theta)] = G(\theta), \quad Tr(Cov_{x\sim\rho}(g_x(\theta))) = \mu(\theta)||G(\theta)||_2^2,$$

where ρ is the data distribution, $\mu(\theta)$ is the scalar that reflects the scale of gradient noise variance introduced by the dataset.

Assumption 2. ***(Local Smoothness and Stability)*** *Within a small neighborhood of θ (short-term dynamics), we assume that the Hessian matrix $H(\theta)$ of the overall loss function and gradient noise scale $\mu(\theta)$ can be approximated as constants:*

$$H(\theta) \approx H, \quad \mu(\theta) \approx \mu.$$

Assumption 3. ***(Independent and Identically Distributed Gradients)*** *The stochastic gradients $\{g_{x_i}(\theta)\}_{i=1}^N$ computed on different data samples $\{x_i\}_{i=1}^N$ are independent and identically distributed (i.i.d.), satisfying:*

$$g_{x_i}(\theta) \overset{i.i.d.}{\sim} \rho_\theta \quad \textit{for all } i = 1, ..., N,$$

where ρ_θ is a distribution characterized by Assumption 1, N is the dataset size.

APPENDIX B
PROOF OF THEOREM 1

Before proving Theorem 1, we first prove Lemma 1.

Lemma 1. *Under the Assumption 1, Assumption 2 and Assumption 3, denote the learning rate by η and the average eigenvalue of the Hessian matrix H of the loss function by $\bar{\lambda}$. Denote the global batch size by B and the gradient noise scale by μ. β is the constant smooth factor defined in Definition 1. The theoretical expected stable value of GR_T is*

$$\mathbb{E}[GR_T] \approx \frac{\left((1+\frac{\mu}{B})^2(1-\bar{\lambda}\eta-\beta)^2\right)\left(\beta^{\frac{1}{1-\beta}} - \left(1-\frac{2\bar{\lambda}\eta}{1+\frac{\mu}{B}}\right)^{\frac{1}{1-\beta}}\right)}{(1-\beta)\left(2\bar{\lambda}\eta-(1-\beta)\left(1+\frac{\mu}{B}\right)\right)\left(\beta^{\frac{1}{1-\beta}} - \left(1-\bar{\lambda}\eta\right)^{\frac{1}{1-\beta}}\right)^2}. \tag{7}$$

Proof. For a given dataset with data points in the distribution $x \sim D$, each data point x at parameter θ is associated with the gradient $g_x(\theta)$. Denote the global batch size as B. Then the sampled batch gradient $G_{est}(\theta)$ at parameter θ is

$$G_{est}(\theta) = \frac{1}{B}\sum_{i=1}^{B} g_{x_i}(\theta); \quad x_i \sim \rho. \tag{8}$$

During the training process, denote the learning rate as η, and the parameter at the t-th iteration be θ_t. It can be considered that

$$\theta_{t+1} = \theta_t - \eta G_{est}(\theta_t). \tag{9}$$

Since the update can be regarded as a small quantity, we perform a Taylor expansion as

$$G_{est}(\theta_{t+1}) = \frac{1}{B}\sum_{i=1}^{B} g_{x_i}(\theta_t - \eta G_{est}(\theta_t)) \approx \frac{1}{B}\sum_{i=1}^{B} g_{x_i}(\theta_t) - \frac{\eta}{B}\sum_{i=1}^{B} H_{x_i}(\theta_t)^T G_{est}(\theta_t), \tag{10}$$

where $H_{x_i}(\theta_t)$ denotes the Hessian matrix of the loss function on data point x_i at parameter θ_t. Since the update is regarded as small quantity and Assumption 2, the Hessian matrix of the different loss function and parameters can be regarded as constant, which is denoted as H. Than we take the expectation of Eq. 8 and Eq. 10 and obtain that

$$\mathbb{E}_{x\sim\rho}[G_{est}(\theta_t)] = G(\theta_t), \tag{11}$$
$$\mathbb{E}_{x\sim\rho}[G_{est}(\theta_{t+1})] = (I - \eta H^T)\mathbb{E}_{x\sim\rho}[G_{est}(\theta_t)]. \tag{12}$$

Adopting the idea of chunking, we set $t = t_0$ as the new time zero point. Let the parameter at new iteration 0 be θ_0. Therefore, at the new iteration t, we have

$$\mathbb{E}_{x\sim\rho}[G_{est}(\theta_t)] = (I - \eta H)^t G(\theta_0). \tag{13}$$

Then we calculate the theoretical expected value of $||\hat{m}_t||_2^2$ which is defined in definition 1. By Eq. 13, it's obvious that

$$\begin{aligned}\mathbb{E}_{x\sim\rho}[\hat{m}_t] \approx& \beta^i \cdot \mathbb{E}_{x\sim\rho}[\hat{m}_0] + (1-\beta)\sum_{k=0}^{t-1}\beta^{t-1-k}\cdot\mathbb{E}_{x\sim\rho}[G_{est}(\theta_k)] \\ =& \beta^i \cdot \mathbb{E}_{x\sim\rho}[\hat{m}_0] + (1-\beta)\beta^{t-1}\left(\sum_{k=0}^{t-1}\left(\frac{I-\eta H}{\beta}\right)^k\right)G(\theta_0).\end{aligned} \tag{14}$$

Assume the eigenvalue decomposition of H is given by $H = P\Lambda P^T$, where P is the orthogonal matrix of eigenvectors and $\Lambda = diag(\lambda_1, \ldots, \lambda_n)$ is the diagonal matrix of eigenvalues. Denote the average eigenvalue of H as $\bar{\lambda} = \frac{1}{n}\sum_{i=1}^n \lambda_i$ and let effective learning rate $\bar{\lambda}\eta = \bar{\lambda}\eta$. Then by transformation and approximate, we know that (we leverage the formula $P = P^T = P^{-1}$ when P is orthogonal matrix, and ignore $\hat{m}_0$ since β^t can regarded as small value as t increasing)

$$\begin{aligned}||\mathbb{E}_{x\sim\rho}[\hat{m}_t]||_2^2 &\approx (1-\beta)^2\beta^{2t-2}G^T(\theta_0)P\left(\sum_{k=0}^{t-1}\left(\frac{I-\eta\Lambda}{\beta}\right)^k\right)^2 P^T G(\theta_0) \\ &\approx (1-\beta)^2\beta^{2t-2}\left(\sum_{k=0}^{t-1}\left(\frac{1-\bar{\lambda}\eta}{\beta}\right)^k\right)^2 G^T(\theta_0)PP^T G(\theta_0) \\ &= \frac{(1-\beta)^2\beta^{2t}}{(1-\bar{\lambda}\eta-\beta)^2}\left(\left(\frac{1-\bar{\lambda}\eta}{\beta}\right)^t - 1\right)^2 ||G(\theta_0)||_2^2.\end{aligned} \tag{15}$$

Next, we analyze the variance. Since the batches selected between iteration t and $t+1$ are independent (which are denoted as x_i and x_j), we can get that (ignoring higher-order terms)

$$\begin{aligned}&G_{est}(\theta_{t+1})G_{est}^T(\theta_{t+1}) \\ \approx& \left(\frac{1}{B}\sum_{i=1}^B g_{x_i}(\theta_t) - \frac{\eta H^T}{B}\sum_{j=1}^B g_{x_j}(\theta_t)\right)\left(\frac{1}{B}\sum_{i=1}^B g_{x_i}(\theta_t) - \frac{\eta H^T}{B}\sum_{j=1}^B g_{x_j}(\theta_t)\right)^T \\ \approx& \frac{1}{B^2}\sum_{i=1}^B g_{x_i}(\theta_t)\sum_{i=1}^B g_{x_i}^T(\theta_t) - \frac{\eta H^T}{B^2}\sum_{j=1}^B g_{x_j}(\theta_t)\sum_{i=1}^B g_{x_i}^T(\theta_t) - \frac{\eta}{B^2}\sum_{i=1}^B g_{x_i}(\theta_t)\sum_{j=1}^B g_{x_j}^T(\theta_t)H.\end{aligned} \tag{16}$$

Then by Assumption 3, we take the expectation

$$\begin{aligned}&\mathbb{E}_{x\sim\rho}\left[G_{est}(\theta_{t+1})G_{est}^T(\theta_{t+1})\right] \\ \approx& \left(\frac{1}{B}\mathbb{E}_{x\sim\rho}[g_x(\theta)g_x^T(\theta)] + \frac{B-1}{B}G(\theta)G^T(\theta)\right) - \eta H^T G(\theta_t)G^T(\theta_t) - \eta G(\theta_t)G^T(\theta_t)H.\end{aligned} \tag{17}$$

Since the covariance matrix is defined as $Cov_{x\sim\rho}(g_x(\theta)) = \mathbb{E}_{x\sim\rho}[g_x(\theta)g_x^T(\theta)] - G(\theta)G^T(\theta)$, we can obtain that

$$Tr\left(\mathbb{E}_{x\sim\rho}\left[G_{est}(\theta_{t+1})G_{est}^T(\theta_{t+1})\right]\right) \approx \frac{1}{B}Tr\left(Cov_{x\sim\rho}(g_x(\theta))\right) + Tr\left(G(\theta_t)G^T(\theta_t)\right) - 2\eta Tr\left(G(\theta_t)G^T(\theta_t)H\right). \tag{18}$$

Similarly, we decompose $H = P\Lambda P^T$, and obtain the results by transformation and approximate (we leverage the formulas $Tr(AB) = Tr(BA)$ and $Tr(PAP^T) = Tr(A)$, where P is the orthogonal matrix)

$$Tr\left(G(\theta_t)G^T(\theta_t)H\right) = Tr\left(G(\theta_t)G^T(\theta_t)P\Lambda P^T\right) = Tr\left(\Lambda P^T\left(G(\theta_t)G^T(\theta_t)\right)P\right) \approx \bar{\lambda}Tr\left(G(\theta_t)G^T(\theta_t)\right). \tag{19}$$

From Assumption 1 and Assumption 2 we know that $Tr(Cov_{x\sim\rho}(g_x(\theta))) = \mu||G(\theta_t)||^2$, where μ is the constant gradient noise scale. Then based on Eq. 18, Eq. 19 and assumptions we have (we leverage the formulas $Tr(xx^T) = ||x||_2^2, x \in R^d$ and $\mathbb{E}[Tr(A)] = Tr\left(\mathbb{E}[A]\right)$)

$$\mathbb{E}_{x\sim\rho}\left[||G_{est}(\theta_{t+1})||_2^2\right] \approx \left(1 - 2\bar{\lambda}\eta + \frac{\mu}{B}\right)||G(\theta_t)||_2^2. \tag{20}$$

Since Eq. 20 holds for any $\bar{\lambda}\eta$, we let $\bar{\lambda}\eta \to 0$ and have $\theta_{t+1} \to \theta_t$. Then we obtain that

$$\mathbb{E}_{x\sim\rho}\left[||G_{est}(\theta_t)||_2^2\right] \approx \left(1 + \frac{\mu}{B}\right)||G(\theta_t)||_2^2. \tag{21}$$

Similarly, we have

$$\mathbb{E}_{x\sim\rho}[||G_{est}(\theta_t)||_2^2] = \left(1 + \frac{\mu}{B}\right)\left(1 - \frac{2\bar{\lambda}\eta}{1 + \frac{\mu}{B}}\right)^t ||G(\theta_0)||_2^2. \tag{22}$$

Also we calculate the theoretical expected value of $||\hat{v}_t||_1$ which is defined in definition 1. By Eq. 22, we can similarly get the result that (also ignore $\hat{v}_0$ due to the same reason)

$$\begin{aligned}
\mathbb{E}_{x\sim\rho}[||\hat{v}_t||_1] &\approx \beta^i \cdot \mathbb{E}_{x\sim\rho}[||\hat{v}_0||_1] + (1-\beta)\sum_{k=0}^{t-1}\beta^{t-1-k}\mathbb{E}_{x\sim\rho}[||G_{est}(\theta_k)||_2^2] \\
&\approx \left(1 + \frac{\mu}{B}\right)(1-\beta)\beta^{t-1}\left(\sum_{k=0}^{t-1}\left(\frac{1 - \frac{2\bar{\lambda}\eta}{1+\frac{\mu}{B}}}{\beta}\right)^k\right)||G(\theta_0)||_2^2 \\
&= \frac{\left(1 + \frac{\mu}{B}\right)^2(1-\beta)\beta^t}{(1-\beta)\left(1 + \frac{\mu}{B}\right) - 2\bar{\lambda}\eta}\left(\left(\frac{1 - \frac{2\bar{\lambda}\eta}{1+\frac{\mu}{B}}}{\beta}\right)^t - 1\right)||G(\theta_0)||_2^2.
\end{aligned} \tag{23}$$

Based on Eq. 15 and Eq. 23, we can get the theoretical expected value of $GR_T(t)$:

$$\mathbb{E}_{x\sim\rho}[GR_T(t)] \approx \frac{\mathbb{E}_{x\sim\rho}[||\hat{v}_t||_1]}{\mathbb{E}_{x\sim\rho}[||\hat{m}_t||_2^2]} \approx \frac{(1 + \frac{\mu}{B})^2(1 - \bar{\lambda}\eta - \beta)^2}{(1-\beta)\left(2\bar{\lambda}\eta - (1-\beta)\left(1 + \frac{\mu}{B}\right)\right)} \cdot \frac{\beta^t - \left(1 - \frac{2\bar{\lambda}\eta}{1+\frac{\mu}{B}}\right)^t}{\left(\beta^t - \left(1 - \bar{\lambda}\eta\right)^t\right)^2}. \tag{24}$$

For t, because EMA processing is used, $t = \frac{1}{1-\beta}$ is used as the estimated time for the expected stable value of GR_T. Thus we obtain that

$$\mathbb{E}_{x\sim\rho}[GR_T] \approx \frac{(1 + \frac{\mu}{B})^2(1 - \bar{\lambda}\eta - \beta)^2}{(1-\beta)\left(2\bar{\lambda}\eta - (1-\beta)\left(1 + \frac{\mu}{B}\right)\right)} \cdot \frac{\beta^{\frac{1}{1-\beta}} - \left(1 - \frac{2\bar{\lambda}\eta}{1+\frac{\mu}{B}}\right)^{\frac{1}{1-\beta}}}{\left(\beta^{\frac{1}{1-\beta}} - \left(1 - \bar{\lambda}\eta\right)^{\frac{1}{1-\beta}}\right)^2}. \tag{25}$$

Here, we complete the proof of Lemma 1. □

Then we proceed to prove Theorem 1.

Theorem 1. *There exists a positive partial derivative relationship between the learning rate $\bar{\lambda}\eta$ and the temporal gradient redundancy GR_T:*

$$\frac{\partial}{\partial\eta}\mathbb{E}[GR_T] > 0. \tag{26}$$

Proof. Based on Lemma 1, we perform a Taylor expansion of GR_T with respect to $\bar{\lambda}\eta$ and approximating to the first order

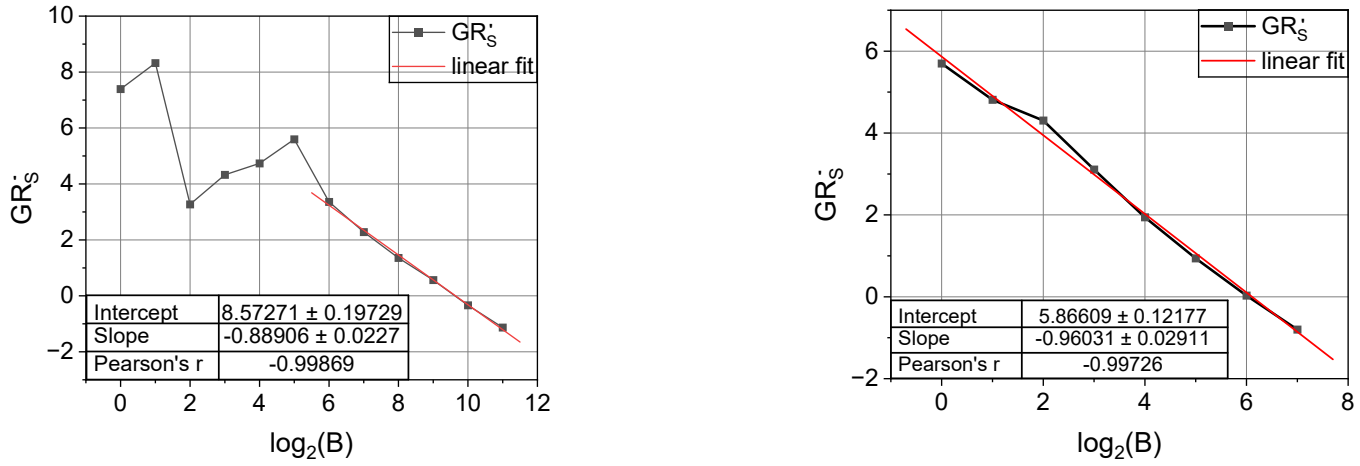
$$\mathbb{E}_{x\sim\rho}[GR_T] \approx \frac{1 + \frac{\mu}{B}}{1 - \beta^t} + \frac{2\frac{\mu}{B}}{1 - \beta^t}\left(\frac{\frac{1}{1-\beta}}{1 - \beta^{\frac{1}{1-\beta}}} - \frac{1}{1-\beta}\right)\bar{\lambda}\eta. \tag{27}$$

Therefore we have

$$\frac{\partial}{\partial\bar{\lambda}\eta}\mathbb{E}_{x\sim\rho}[GR_T] \approx \frac{2\frac{\mu}{B}\bar{\lambda}}{1 - \beta^t}\left(\frac{\frac{1}{1-\beta}}{1 - \beta^{\frac{1}{1-\beta}}} - \frac{1}{1-\beta}\right). \tag{28}$$

Since $0 < \beta < 1$, for $t \in \mathbb{N}_+$, the following inequality holds

$$\frac{i}{1 - \beta^t} - \frac{1}{1-\beta} = \frac{t - (1 + \beta + \cdots + \beta^{t-1})}{(1-\beta)(1 + \beta + \cdots + \beta^{t-1})} > 0. \tag{29}$$



(a) ResNet18 (b) Bert

Fig. 13: Verification of the linear relationship between GR_S and $\log_2(B)$ on different models.

Through monotonicity, we know the inequality holds when $i = \frac{1}{1-\beta}$. Now we obtain that

$$\frac{\partial}{\partial \eta}\mathbb{E}_{x\sim\rho}[GR_T] > 0. \tag{30}$$

Here, we complete the proof of Theorem 1. □

Appendix C
Proof of Theorem 2

Before proving Theorem 2, we first prove Lemma 2.

Lemma 2. *Under the Assumption 1, Assumption 2 and Assumption 3, denote the global batch size by B and the gradient noise scale by μ. The theoretical expected stable value of GR_S is*

$$\mathbb{E}\left[GR_S\right] \approx \frac{1+\mu}{B+\mu}. \tag{31}$$

Proof. By Assumption 1 and Assumption 2, we have (we leverage the formula $\mathbb{E}[||x^2||_1] = Tr(Cov_{x\sim\rho}(x))$ for $x \in R^d$):

$$\mathbb{E}_{x\sim\rho}\left[\left|\left|\sum_{i=1}^{B} g_{x_i}^2(\theta_t)\right|\right|_1\right] = B\cdot Tr\left(Cov_{x\sim\rho}(g_{x_i}(\theta_t))\right) + B||G(\theta_t)||_2^2 = B(\mu+1)||G(\theta_t)||_2^2. \tag{32}$$

Based on Assumption 3, we know that $g_{x_i}(\theta_t)$ and $g_{x_j}(\theta_t)$ are independent and identically distributed for $i \neq j$, i.e., $\mathbb{E}_{x\sim\rho}[g_{x_i}^T(\theta_t)g_{x_j}(\theta_t)] = (\mathbb{E}_{x\sim\rho}[g_x(\theta_t)])^T(\mathbb{E}_{x\sim\rho}[g_x(\theta_t)]), \forall i \neq j$. Therefore we can obtain that

$$\mathbb{E}_{x\sim\rho}\left[\left|\left|\sum_{i=1}^{B} g_{x_i}(\theta_t)\right|\right|_2^2\right] = \mathbb{E}_{x\sim\rho}\left[\left|\left|\sum_{i=1}^{B} g_{x_i}^2(\theta_t)\right|\right|_1\right] + \mathbb{E}_{x\sim\rho}\left[\sum_{1\leq i,j\leq B, i\neq j} g_{x_i}^T(\theta_t)g_{x_j}(\theta_t)\right] = \left(B\mu + B^2\right)||G(\theta_t)||_2^2. \tag{33}$$

By definition 2, we have

$$\mathbb{E}_x[GR_S] \approx \frac{\mathbb{E}_{x\sim\rho}\left[\left|\left|\sum_{i=1}^{B} g_{x_i}^2(\theta_t)\right|\right|_1\right]}{\mathbb{E}_{x\sim\rho}\left[\left|\left|\sum_{i=1}^{B} g_{x_i}(\theta_t)\right|\right|_2^2\right]} = \frac{1+\mu}{B+\mu}. \tag{34}$$

□

In Fig. 13, we further validate Lemma 2 at the beginning of the training process. We measure the GR_S across varying batch sizes for both ResNet18 and BERT, which are quintessential models representing the CNN and transformer architectures, respectively. For ease of presentation, we define $GR_S^{'} = \log_2 \frac{GR_S}{1-GR_S}$. According to Lemma 2, we have $\mathbb{E}_{x\sim\rho}[GR_S^{'}] = \log_2 \mu - \log_2 B$. As

shown in Fig. 13, a strong linear relationship between $GR_S^{'}$ and $\log_2(B)$ is observed. By performing linear fitting, the slope is found to be close to the theoretical value of -1, which confirms the correctness of this relationship[6].

We next proceed to prove Theorem 2.

Theorem 2. *There exists a positive partial derivative relationship between the batch size B and the temporal gradient redundancy GR_S:*

$$\frac{\partial}{\partial B}\mathbb{E}[GR_S] < 0. \tag{35}$$

Proof. By Lemma 2, it's obvious that

$$\frac{\partial}{\partial B}\mathbb{E}[GR_S] = -\frac{1+\mu}{(B+\mu)^2} < 0. \tag{36}$$

□

APPENDIX D
THE METHODS FOR PRACTICAL GR_S MEASUREMENTS

Notice that it is impractical to compute the gradient for each sample individually. In data parallelism, we assume there are n devices, so the local batch size is set to B/n. Let the gradient aggregated within each device i be denoted as $\tilde{G}_{est,i}(\theta_t)$. Similar to GR_S, we define

$$\xi_k = \frac{||\sum_{i=1}^{k}\tilde{G}^2_{est,i}(\theta_t)||_1}{||\sum_{i=1}^{k}\tilde{G}_{est,i}(\theta_t)||_2^2} \tag{37}$$

for $k = 1, 2, \ldots, n$. Note that in data-parallel distributed training, we perform an all-reduce operation every iteration. This process involves collecting the gradients from all local batches and then reducing them. Through this process that we can easily compute $\sum_{i=1}^{k}\tilde{G}^2_{est,i}(\theta_t)$ and $\sum_{i=1}^{k}\tilde{G}_{est,i}(\theta_t)$ between devices when reducing the gradient, thus the calculation of ξ_k is feasible. According to Assumption 1 and Assumption 3, we can similarly obtain that

$$Tr(Cov_{x\sim\rho}(\tilde{G}_{est}(\theta))) = \frac{Tr(Cov_{x\sim\rho}(g_x(\theta)))}{B/n} = \frac{n}{B}\mu||G(\theta)||_2^2. \tag{38}$$

Similarly to Appendix C, we can easily obtain that

$$\mathbb{E}_{x\sim\rho}\left[\xi_k\right] \approx \frac{1+\frac{n\mu}{B}}{k+\frac{n\mu}{B}}. \tag{39}$$

Therefore, by transformation, we have

$$\frac{k-1}{1-\xi_k} - k = \frac{n}{B}\mu. \tag{40}$$

By performing linear fitting between $\left(\frac{k-1}{1-\xi_k} - k\right)$ and $\frac{n}{B}$ (for $k = 1, 2, \ldots, n$), the slope μ can be obtained when $n \geq 2$, from which $\mathbb{E}_x\left[GR_S\right]$ can be further derived from Lemma 2 which has been solidly validated. In the special case when $n = 1$ (only a single worker), we set the gradient accumulation steps to at least two, through which equivalent estimates across multiple devices can be obtained.

APPENDIX E
THEORETICAL ESTIMATE OF THE IDEAL VALUE FOR GR_T AND GR_S

We first estimate the theoretical ideal value for GR_S. Previous works [45] suggests that unlimitedly increasing the batch size does not yield proportional performance scaling. There exists a 'turning point' beyond which further batch size enlargement leads to rapidly diminishing scaling returns. The turning point is given as

$$\mathcal{B} = \frac{Tr(H \cdot Cov(\nabla L_i))}{g^T H g} \approx \frac{\mu\bar{\lambda}||g||_2^2}{\bar{\lambda}||g||_2^2} = \mu. \tag{41}$$

This suggests that batch size should not exceed μ to avoid helpless and inefficient scale up. According to Lemma 2, we get the corresponding GR_S value with $B = \mathcal{B}$ is

$$\mathbb{E}_{x\sim\rho}[GR_S]_{B=\mathcal{B}} \approx \frac{1+\mu}{B+\mu} \approx 0.5. \tag{42}$$

Since $\frac{\partial}{\partial B}\mathbb{E}_{x\sim\rho}[GR_S] < 0$, this suggests that the ideal $\mathbb{E}_{x\sim\rho}[GR_S] \geq 0.5$.

[6]It is noted that in ResNet18, when the batch size is set too small, the relationship breaks down due to the excessive variance and significant measurement fluctuations caused by the minimal batch size. However, this range is not within our area of interest. In the range we are concerned with, Lemma 2 holds strongly and remains valid.

On the other hand, to achieve efficient training, the batch size should not be too small. A typical lower-bound estimate is half of the turning point batch size (i.e., $\mathcal{B}/2$). Under this configuration, when applied to Lemma 2, we obtain $\mathbb{E}_{x\sim\rho}[GR_S] = \frac{1+\mu}{\mathcal{B}/2+\mu} \approx 0.65$. In summary, we theoretically estimate the GR_S range to be $[0.5, 0.65]$, which has been empirically validated by our preceding experiments.

Then we estimate the theoretical ideal value for GR_T. We take $B = \mu$ as the typical value. According to the gradient noise scale model [45], it gives the best learning rate with best efficiency by

$$\eta_{opt} = \frac{||g||_2^2}{g^T H g} \frac{1}{1+\frac{\mathcal{B}}{B}} \approx \frac{1}{2\bar{\lambda}}. \tag{43}$$

From Lemma 1, we obtain that

$$\mathbb{E}_{x\sim\rho}[GR_T]_{B=\mu,\beta=0.9,\eta=\frac{1}{2\bar{\lambda}}} = \left[\frac{(1+\frac{\mu}{B})^2(1-\bar{\lambda}\eta-\beta)^2}{(1-\beta)\left(2\bar{\lambda}\eta-(1-\beta)\left(1+\frac{\mu}{B}\right)\right)} \cdot \frac{\beta^{\frac{1}{1-\beta}} - \left(1-\frac{2\bar{\lambda}\eta}{1+\frac{\mu}{B}}\right)^{\frac{1}{1-\beta}}}{\left(\beta^{\frac{1}{1-\beta}} - \left(1-\bar{\lambda}\eta\right)^{\frac{1}{1-\beta}}\right)^2} \right]_{B=\mu,\beta=0.9,\eta=\frac{1}{2\bar{\lambda}}} \approx 20.$$

That suggests the ideal value of GR_T will be around 20.

APPENDIX F
DISCUSSIONS

Applicability in diverse optimizer (e.g., momentum, weight decay) setups. Data samples are the sources of model improvement; it is a fundamental objective to efficiently exploit the data samples; any optimizer, momentum or weight-decay method (like [58], [29], [41]) is merely a *measure* pursuing that objective. Our GR metric is calculated out of the original gradient, prior to the transformations imposed by optimizers. Such vanilla gradient provides a general angle reflecting how samples (by their temporal/spatial voting consistency) are exploited in make training contributions. In that sense, a healthy redundancy state is a *shared objective* for any optimizer (*different optimizers essentially only express different insights on how to reach that healthy state*). Therefore, by defining GR and also providing a guiding target, we provide a *general feedback-control paradigm* that can be leveraged by any optimizer/policy designers. In fact, it can be viewed that AdaGrad and Adam (careful readers may have noticed that their adaptivity formulas are similar to our GR_T definition) are covered by such a paradigm. In summary, GR-Gauge essentially describes a *general objective*, and existing optimization methods are *specific measures*. Under our feedback-control paradigm, we can well absorb (even rectify) the effect of other hyper-parameter optimizing actions. That said, on the other hand, we admit that it is still risky to indiscriminately enforce our GR-Gauge target, given the complexity of training processes. That is why we modestly limit GR-Gauge application to *critical periods* (so that we can join GR-Gauge with other excellent online-tuning methods) and to *cost-sensitively users* (so that slight accuracy degradation is bearable if occurs).

Hyperparameter scope. This work focuses on learning rate and batch size because they exhibit stable and actionable correlations with GR_T and GR_S, respectively. GR-Gauge does not directly address broader AutoML problems such as neural architecture search [39], where candidate configurations alter the model structure itself. Nevertheless, the gradient-redundancy perspective may potentially be extended to other hyperparameters that affect training dynamics, such as momentum [58] and synchronization interval [61]. Such an extension requires establishing a measurable and stable relationship between the corresponding hyperparameter and temporal or spatial gradient redundancy.

Integration with existing cluster schedulers. GR-Gauge is complementary to cluster-level resource schedulers. Existing schedulers address queuing delay, gang scheduling, placement constraints, resource utilization, and fairness [26], [19], [64], [43], [49], while recent systems further coordinate training dynamics with resource allocation [52], [37]. GR-Gauge does not replace these scheduling mechanisms. Instead, it adjusts the target GR_S so that a job can expose different batch-size and GPU-demand operating points; the underlying scheduler remains responsible for job admission and placement.

APPENDIX G
IMPLEMENTATION

We implement GR-Gauge in PyTorch with about 2,000 lines of code. Although GR-Gauge avoids frequent and costly validation, calculating the gradient-redundancy metrics still introduces additional overhead. To control this overhead, we adopt two design choices: *per-device gradient estimation* and *parameter sampling*, which we detail below.

Per-device estimation of GR_S. Computing the exact GR_S requires per-sample gradients, which would introduce additional backward passes. In data-parallel training, however, each GPU already computes an aggregated gradient over its local mini-batch before the all-reduce operation. GR-Gauge reuses these per-device gradients to estimate spatial redundancy at the global-batch level, avoiding the explicit computation of individual-sample gradients. This approximation preserves the trend of GR_S while substantially reducing its measurement overhead. The detailed estimator and its justification are provided in Appendix D.

Parameter sampling. GR-Gauge calculates GR_T and GR_S over a sampled subset of model parameters rather than the entire model. Specifically, it samples at most $L = 5 \times 10^7$ parameters, with the number of sampled parameters from each layer proportional to the layer size. This design is motivated by prior observations that parameters within the same layer generally exhibit similar gradient statistics [31], [42]. Layer-proportional sampling therefore preserves the representativeness of the measured gradient statistics while reducing both computation and memory overhead.

APPENDIX H
ADDITIONAL RELATED WORK

Advanced HPO systems. Beyond the classical HPO methods discussed in Sec. II-B, recent systems improve HPO from different perspectives, including ultra-large-scale training configuration [35], [67], distributed HPO execution [38], federated HPO [68], human-centered interactive HPO [18], and trial-data reduction [28]. These studies mainly optimize how candidate configurations are generated, evaluated, or scheduled. In contrast, GR-Gauge exposes gradient redundancy as an internal training-health signal to guide learning-rate and batch-size search, early termination, and state reuse, complementing existing HPO frameworks.

Hyper-gradient based adaptive training configuration. Hyper-gradient methods update hyper-parameters by differentiating the validation objective or training dynamics [50], [47]. They are designed for long-term adaptation, and may require additional hyper-gradient or Hessian-related computation. Different from these methods, GR-Gauge focuses on promptly identifying a healthy learning-rate and batch-size configuration during the (early-stage) critical training period, without incurring substantial computation overhead.

Training job scheduling systems. Shared GPU clusters may suffer from queuing delay and resource under-utilization due to gang scheduling, placement constraints, and coarse-grained GPU allocation [26]. Existing cluster schedulers optimize job completion time, utilization, fairness, and heterogeneous resource allocation [19], [64], [43], [49]. Recent systems further coordinate training dynamics with resource allocation, for example by jointly optimizing statistical efficiency and system throughput or reallocating resources among HPO trials [52], [37]. Nonetheless, these systems primarily operate at the scheduling level, either taking the initial job resource demand as given or adapting it inside the scheduler; by contrast, our resource-aware HPO extension adapts batch size and per-job GPU demand in HPO, which can help yield higher overall cluster efficiency.